DESIGN OF VLSI-BASED HARDWARE DETECTION SYSTEM AGAINST POWER SIDE-CHANNEL ATTACKS

A PROJECT REPORT

Submitted by

ASHWIN KUMAR A	**210922106002**
GOWTHAM S	**210922106014**
SANJAY C	**210922106042**

in the partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

LOYOLA INSTITUTE OF TECHNOLOGY

PALANCHUR, CHENNAI-600123

ANNA UNIVERSITY : CHENNAI 600025

MAY 2026

ANNA UNIVERSITY : CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report **"DESIGN OF VLSI-BASED HARDWARE DETECTION SYSTEM AGAINST POWER SIDE-CHANNEL ATTACKS"** is the bonafide work of **"ASHWIN KUMAR A" (210922106002), "GOWTHAM S" (210922106014) and** "**SANJAY C" (210922106042)** who carried out the project work under our supervision.

SIGNATURE

Mrs.R.UMAMAHESWARI M.E.,(Ph.D.),
HEAD OF THE DEPARTMENT
Department of Electronics and Communication Engineering,
Loyola Institute of Technology,
Palanchur, Chennai-600123.

SIGNATURE

Dr. V.PUSHPA M.E., Ph.D.,
SUPERVISOR
ASSISTANT PROFESSOR,
Department of Electronics and Communication Engineering,
Loyola Institute of Technology,
Palanchur, Chennai-600123.

Anna University Practical Project **Viva-Voce** held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We are immensely pleased to take up this opportunity to thank the **LORD ALMIGHTY** for showering his unlimited blessing upon us.

We take this opportunity to express our gratitude to our Founder and Chairman **Rev.Fr. Dr.J.E.ARULRAJ**, our Correspondent **Rev.Sr. Dr. M.K.TERESA** and our Administrator **Rev.Sr. K.NAMBIKAI MARY** for allowing us to take over this project.

We thank our Principal **Dr.P.SEENI KANNAN M.E., Ph.D.**, who has always served as an inspiration for us to carry out our responsibilities also providing approval for doing this project.

We express our thanks to **Mrs. R.UMAMAHESWARI M.E., (Ph.D.),** Head of the Department and Project Co-ordinator for the scintillating discussion and encouragement towards our project.

We express our gratitude to our Supervisor **Dr. V.PUSHPA M.E., Ph.D.**, Assistant Professor, for her constant guidance and co-operation during the project work.

It is a pleasure to acknowledge our indebtedness to all the **Faculty of the Department of Electronics and Communication Engineering** who aided us successfully to bring our project a effective one. Further thanks to **our Parents, our Family members and Friends** for their moral support.

ASHWIN KUMAR A

GOWTHAM S

SANJAY C

ABSTRACT

The growth of SOC devices in the latest technologies going high every day. Powerful side-channel analysis (SCA) attacks based on failure analysis (FA) techniques can bypass conventional countermeasures on integrated circuits (ICs), and therefore, break the entire system's security. Laser Logic State Imaging (LLSI) from the IC backside is an example of such attacks, making the contactless probing of static on-die signals possible. Several countermeasures have been proposed to prevent optical probing attacks, such as LSI. However, these schemes are designed according to the laser properties and its impact on transistors, and hence, they have complex fabrication steps and large area overhead. As a result, they are difficult to verify and implement. In this paper, we propose a twofold detection self-timed sensor, which is the first attempt, to our knowledge, for an easy-to implement circuit-based countermeasure to block LLSI attacks. To perform LLSI, the attacker needs to freeze the clock at a point of interest and modulate the voltage supply line at a known frequency to leak the state of transistors through laser light reflections. With these two attack requirements in mind, we design, simulate, and implement clock- and voltage-based sensors that can detect VLSI attacks with very high confidence. This approach enhances hardware security by providing a low-overhead, scalable, and practical solution for real-time attack detection in SoC systems. Furthermore, the proposed detection mechanism can be seamlessly integrated into existing SoC architectures without significant performance degradation, making it suitable for real-world applications. Its adaptability to evolving attack strategies ensures long-term reliability, while enabling designers to achieve a balanced trade-off between security, area overhead, and power efficiency in advanced VLSI systems.

CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

AES	:	Advanced Encryption Standard
ASIC	:	Application-Specific Integrated Circuit
BDD	:	Binary Decision Diagram
BIST	:	Built-In Self-Test
CDC	:	Clock Domain Crossing
CLI	:	Command-Line Interface
CPA	:	Correlation Power Analysis
CS	:	Cone-Size
DFT	:	Design For Testability
DPLL	:	Digital Phase-Locked Loop
DRC	:	Design Rule Check
DSP	:	Digital Signal Processing
EDA	:	Electronic Design Automation
ERC	:	Electrical Rule Checking
FSM	:	Finite State Machine
FPGA	:	Field Programmable Gate Array
GUI	:	Graphical User Interface
HD	:	Hamming Distance
HDL	:	Hardware Description Language
IBM	:	International Business Machines
IC	:	Integrated Circuit
IP	:	Intellectual Property
IR	:	Infrared
JTAG	:	Joint Test Action Group
KSA	:	Key Sensitization Attacks
LVS	:	Layout Vs. Schematic
NIST	:	National Institute of Standards and
PCB	:	Printed Circuit Board
PLL	:	Phase-Locked Loop

PSC	**:**	Power Side-Channel
RPG	**:**	Random Permutation Generator
RPV	**:**	Recurrent Pattern Verification
RSA	**:**	Rivest–Shamir–Adleman
RTL	**:**	Register Transfer Level
SCA	**:**	Side-Channel Attack Technology
SFLL	**:**	Stripped-Functionality Logic Locking
SLL	**:**	Secure Logic Locking
SoC	**:**	System On Chip
SRAM	**:**	Static Random Access Memory
STA	**:**	Static Timing Analysis
SVA	**:**	System Verilog Assertions
TCL	**:**	Tool Command Language
TSMC	**:**	Taiwan Semiconductor Manufacturing Company
TV	**:**	Television
TVLA	**:**	Test Vector Leakage Assessment
UMC	**:**	United Microelectronics Corporation
VHDL	**:**	VHSIC Hardware Description Language
VLSI	**:**	Very Large Scale Integration

CHAPTER 1

INTRODUCTION

1.1 AIM

The aim of this project is to design and implement an efficient hardware-based framework for detecting and analysing low-power side-channel and physical attacks in modern System-on-Chip (SoC) architectures. The project focuses on developing a robust detection mechanism using FPGA-based design and VLSI simulation techniques to identify anomalies caused by attacks such as voltage manipulation, clock tampering, and side-channel leakage. The objective is to enhance the security and reliability of integrated circuits by incorporating real-time monitoring and attack detection modules.

1.2 OBJECTIVES

The key objectives of this project are as follows:

- To study and analyse various physical and side-channel attacks affecting SoC and FPGA-based systems.
- To design and implement reference clock generation and configurable clock synthesizer modules.
- To develop attack pattern generators capable of simulating multiple attack scenarios such as side-channel attacks, replay attacks, and corruption attacks.
- To implement a dynamic pattern matching algorithm for accurate detection and classification of attacks.
- To integrate multiple modules into a unified FPGA-based system for real-time monitoring and validation.
- To evaluate system performance based on parameters such as power consumption, latency, and area utilization.
- To simulate and verify the complete system using ModelSim for functional and timing validation.
- To enhance the robustness of the system by incorporating efficient hardware-level countermeasures against emerging side-channel and physical attacks. This includes improving detection accuracy and minimizing false positives.

1.3 PURPOSE

The purpose of this project is to address the growing security challenges in modern SoC designs by providing an effective and scalable hardware-based attack detection solution. With the increasing complexity and integration of digital systems, traditional security measures are often insufficient against sophisticated physical and side-channel attacks. This project aims to bridge this gap by introducing a simulation-driven approach that enables early detection of vulnerabilities and improves system resilience. Furthermore, the proposed framework facilitates the development of secure hardware systems by enabling designers to analyze attack impacts and implement appropriate countermeasures during the design phase itself.

1.4 VULNERABILITIES AND ATTACKS

FPGA Market Model

We consider the following major parties in the FPGA-based systems market:

FPGA vendors: These are the companies (like Xilinx and Intel) that design FPGA architecture and build FPGA chips. These chips normally come with some built-in functional units, memory blocks, and other IP components that the FPGA vendors believe the customers will need.

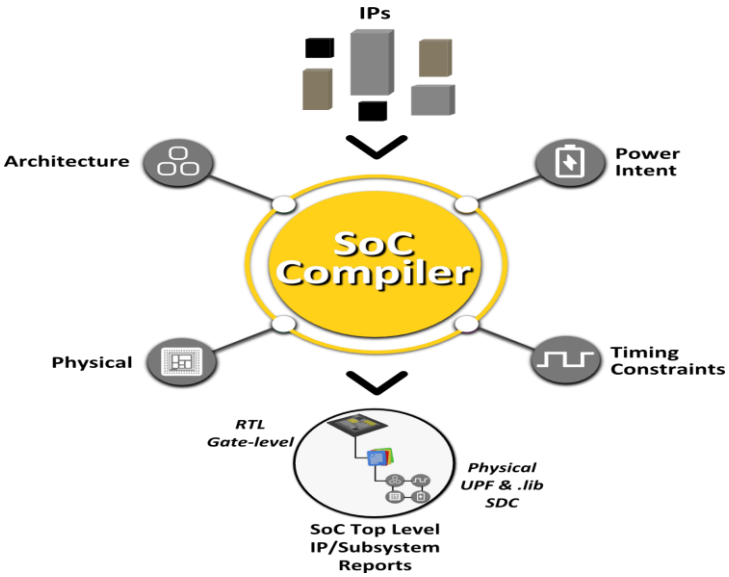

Fig. 1.1 System Architecture of SoC-Based Attack Detection Framework

Foundries: These are the semiconductor manufacturers (like TSMC, UMC, Global foundries, IBM) that will fabricate the FPGA chips for the FPGA vendors.

FPGA-based system developers: These are the companies that create commercial products on FPGA chips. The product is normally in the form of configuration bit stream file for a given family FPGA chips.

FPGA-based system end users: These are companies or individuals who obtain the FPGA-based systems (like network routers, TV set-top boxes) from the system developer and use these systems.

IP core vendors: These are the companies that develop IP cores (like memory blocks, DSP cores) for specific applications, not necessary for FPGA-based systems.

EDA tool vendors: These are the companies that develop software tools to facilitate the design of large-scale integrated circuits, including FPGA chips.

1.4.1 Security and Trust Vulnerabilities

In this section, we analyze the vulnerabilities at each interaction between a pair of parties in the above FPGA-based systems. Same notion (e.g. hardware Trojan or IP protection) may be used for similar vulnerabilities under different scenarios. The general guideline is that on the sending side of an arrow, the party that sends out the request will have concerns about the security of the information or product he sends out; on the receiving end of an arrow, the party needs to verify whether the received service or product is trusted.

FPGA vendors and foundries:

(a) Overbuilding: an untrusted foundry may fabricate more FPGA chips than required and sell them at a lower price to system developers.

(b) Hardware Trojan: during the fabrication process, unwanted functionalities, known as hardware Trojan, may be embedded in the FPGA chips.

(c) Information leaking: FPGA vendor's confidential data needed for the fabrication of the FPGA chips may be mishandled or leaked to parties (e.g. another competing FPGA vendor) that should not have access to such data. This can lead to intellectual property theft, by compromising the security and trustworthiness of the FPGA devices.

FPGA vendors (or system developers) and IP core vendors (or EDA tool vendors):

(a) Hardware Trojan: FPGA vendors (or system developers) need to ensure the IPs provided by IP vendors do not contain malicious Trojans.

(b) Information leaking: malicious programs in EDA tools may take advantage of the Xilinx FPGA design suite's backdoor to manually disturb the FPGA design after placement and routing or collect valuable data about FPGA chip and/or the system.

(c) IP protection: IP core vendors and EDA tool vendors want that their IPs and tools are used and royalty are paid properly.

(d) Reverse engineering: IP core vendors expect their IPs cannot be reverse engineered by untrusted parties.

FPGA vendors and system developers:

(a) Hardware Trojan: : FPGA vendors (or system developers) need to ensure the IPs provided by IP vendors do not contain malicious Trojans.

(b) Information leaking: there is no guarantee that FPGA chips do not contain hardware Trojans and the EDA tools do not have malicious codes. However, system developers have limited options and have to trust the FPGA chips and the associated design tools to design the systems.

(c) System developers and end users: both parties involved in this interaction need to protect their respective IPs. System developers face several security challenges unique to FPGA based system.

(d) Reverse engineering: system developers expect their products cannot be reverse engineered by untrusted parties;

(e) Cloning: the FPGA configuration bitstream is obtained by eavesdropping or from the volatile SRAM and used to configure FPGA chips;

(f) Side-channel attacks: side-channel attacks exploit the leakage of physical information when the encryption algorithm is being executed on a system . These attacks analyze power, timing, or electromagnetic emissions to reveal secret cryptographic keys.

For example, when bitstream file is encrypted as offered by most FPGA vendors, side-channel attacks can reveal the keys stored in the FPGA chips and make the bitstream file unprotected;

(g) FPGA replay attack: an adversary downgrades an FPGA-based system to the previous version of FPGA chips with known vulnerabilities and explores such vulnerabilities.

1.4.2 REPLAY ATTACKS

The replay attacks are particularly dangerous for FPGA-based system security because attackers can effectively preclude security-critical updates by replaying the previous FPGA configurations. Current FPGA IP protection techniques do not consider the replay attack. As shown in Figure, the system developers usually need to update their FPGA-based products Bi to a new version Bj for the sake of upgrading such as fixing the vulnerabilities or eliminating existed hardware Trojans to protect them against security threat, which gives the attackers the chance to downgrade the system into its previous old version Bi so that they can exploit the outdated vulnerabilities to steal secret information or trigger hardware Trojans. For instance, if system developer has detected a hardware Trojan in purchased IP, trustable IP is needed to replace the suspicious IP and even it's necessary to exploit self-developed IP in some sensitive areas. Therefore, it is urgent to provide effective defenses to resist replay attack. Since FPGA itself cannot distinguish old or new bitstream, when a new version is issued, the previous version still can work on FPGA although the bitstream

A. Background on Obfuscation Techniques

1) Types of Obfuscation Key Gates: There are multiple locking mechanisms that have been introduced in the literature. When the circuit is in locked mode, these locking mechanisms aim to either invert the value of the obfuscated net or replace it with another function. The following section shows a detailed overview of the main locking mechanism types used to obfuscate the IP.

a) XOR/XNOR gates: In this type, an inverting mechanism using XOR/XNOR gate is inserted with a key signal to one of the inputs. This mechanism is introduced in as the first functional obfuscation approach. This approach enhances security by controlling output behavior through key inputs, making reverse engineering and unauthorized access significantly more difficult.

b) Multiplexers: A rerouting mechanism using multiplexers is inserted [4], where one input is the original route, and the other input is set as either a dummy function or a different fan-in logic cone in the circuit. Binary decision diagram (BDD) obfuscation is an example of a rerouting mechanism, where the dummy function has a modified copy of the original's BDD. The limitation of this approach is scalability as the overhead grows exponentially with the key size. This rapid growth in overhead occurs because a modified version of the original BDD is duplicated with each key input. Another introduced rerouting technique is called Full-Lock, which focuses on redirecting a set of carefully selected nodes using configurable switch-box modules; these modules consist of a set of rerouting elements that allow the correct routing configuration when the correct key is applied. Moreover, a unique locking technique called cyclic obfuscation utilizes a looping structure that allows for wrong keys to form combinational loops. Cyclic locking has been enhanced using SR Clock, where looping modules exponentially increase the number of routes that create the corrupting feedback loops.

c) Functional-stripping operations: A more advanced locking mechanism uses a functional-stripping module to lock the function of the IP. This module is followed by a functional restoring unit that will allow normal behavior when the correct key is applied. The stripped-functionality logic locking (SFLL) introduced in is an example of this technique.

2) Key-Gate Insertion Strategies: As different locking mechanisms provide various security and performance advantages, the process of selecting a suitable location for the implemented locking mechanism is crucial. The following list gives an overview of the popular key-gate insertion strategies.

a) Random insertion: The insertion process in this approach is nondeterministic and random [1]. This process is scalable to large circuits as no functional or structural circuit analysis is required. Although this approach is flexible and lightweight, it does not guarantee strong functional or structural alteration of the original design.

b) Secure logic locking (SLL): The key-gate insertion strategy introduced in [10] has been implemented to mitigate sensitization attacks. The insertion process analyzes the surroundings of candidate locations to avoid mutable key gates' formation. Mutable key gates are defined as gates that mask other inserted key gates, which can help leak the correct

key values or reduce the key-search space. Therefore, SLL is developed to offer high resiliency against key sensitization attacks (KSAs). However, the process is not scalable to large circuits as the complexity of the introduced algorithm grows exponentially with respect to the circuit size.

c) Logic cone-size (CS): The logic CS insertion strategy is based on coverage analysis of each net in the circuit. The sum of gates in fan-in and fan-out logic cones is calculated for each net, and the nets with the highest values are picked for key-gate insertion. CS focuses on placing key gates in locations that affect most of the circuit, which can help increase the level of output corruption when the incorrect key is applied. The main limitation of this approach is the localized key insertion, as the logic cone analysis shows that most candidate locations are either close to the primary inputs or outputs. Therefore, the structural impact of CS is low, which leaves the circuit vulnerable to structural-based attacks.

d) Controllability-based selection : The key-gate insertion strategy introduced in aims to locate nets with the lowest switching activities. This approach aims to increase the controllability of low activity nets in the circuit. The goal of this approach is to provide a locking mechanism that can protect unauthorized use of the IP and help prevent Trojan insertion.

e) Observability-based selection (Qual): The insertion strategy introduced in aims to select net locations that provide a predictable level of output corruption. This approach (called Qual) analyzes the probability of a net being observable to a primary output and measures the loss of quality of service when the analyzed net is locked. The strategy is used to tolerate a small number of bit-flips in the key when the key source is unreliable.

f) **Fault-impact-based selection (FIS):** This key-gate insertion strategy focuses on identifying nets whose faults cause maximum disruption to circuit functionality. It evaluates the impact of potential faults on output behavior by performing fault simulation and selecting nodes that propagate errors to multiple outputs. By inserting key gates at these high-impact locations, the approach ensures significant degradation in functionality when an incorrect key is applied. This enhances resistance against both structural and functional attacks. However, the method may involve higher computational complexity due to extensive fault analysis, making it less efficient for very large-scale circuits.

CHAPTER 2

LITERATURE SURVEY

The literature survey focuses on recent advancements in side-channel attack (SCA) analysis and corresponding hardware-level countermeasures, particularly in the context of post-quantum cryptographic algorithms such as CRYSTALS-Kyber. Recent studies highlight the increasing vulnerability of even protected hardware implementations, where advanced techniques such as deep learning-assisted power analysis can successfully extract secret keys with high accuracy. These works reveal that conventional protection mechanisms like masking are not entirely secure, as attackers can exploit weaknesses in specific stages such as message decoding during the decryption process. Furthermore, profiled and non-profiled attack models demonstrate the practical feasibility of recovering shared keys through repeated operations, emphasizing the need for stronger and more reliable hardware security mechanisms.

In addition to identifying vulnerabilities, the reviewed works also propose efficient and hardware-friendly countermeasures such as power-aware synthesis, operation shuffling, and architectural modifications to reduce information leakage and improve system resilience.

2.1 LITERATURE REVIEW

2.1.1 A Side-Channel Attack on a Masked Hardware Implementation of CRYSTALS-Kyber[1] - Journal of Cryptographic Engineering.

Yanning Ji and Elena Dubrova (July, 2025)

This paper presents the first side-channel attack targeting a protected hardware implementation of **CRYSTALS-Kyber**, a post-quantum key encapsulation mechanism selected for standardization by NIST. The researchers demonstrate a practical message recovery attack on a first-order masked FPGA implementation of **Kyber-512** using power analysis assisted by deep learning. By exploiting a vulnerability in the masked message decoding function during the decryption step, the attack can achieve a **99.7% success rate** for full shared key recovery through multiple decapsulation repetitions. Additionally, the authors evaluate the feasibility of attacks during encapsulation and propose an architectural

countermeasure—replacing standard registers with **shift registers**—to significantly reduce signal-to-noise ratios and mitigate such hardware-level leakages. The study further highlights the limitations of conventional masking techniques in securing post-quantum cryptographic implementations against advanced side-channel attacks. By leveraging deep learning models, the attackers are able to efficiently learn leakage patterns and correlate them with secret-dependent operations, significantly improving attack effectiveness. The experimental validation on FPGA platforms demonstrates the real-world feasibility of such attacks, emphasizing the urgent need for more robust and adaptive hardware countermeasures to ensure secure deployment of CRYSTALS-Kyber in practical applications.

2.1.2 PoSyn: Secure Power Side-Channel Aware Synthesis[2] - arXiv preprint, arXiv.

Amisha Srivastava, Samit S. Miftah, Hyunmin Kim, Debjit Pal, and Kanad Basu (June, 2025)

This paper introduces **PoSyn**, a novel logic synthesis framework designed to enhance the resistance of cryptographic hardware against **Power Side-Channel (PSC)** attacks. Unlike traditional countermeasures like masking, which often suffer from high area overhead and vulnerability to optimization removal, PoSyn strategically modifies the mapping of Register Transfer Level (RTL) components to standard cells to minimize leakage without altering design functionality. The framework employs an optimal bipartite matching approach guided by a cost function that balances driving strength, capacitance, and switching activity. Evaluated on benchmarks including AES, RSA, and post-quantum algorithms like **CRYSTALS-Kyber**, PoSyn demonstrates significant security improvements, reducing attack success rates to as low as 3%. Furthermore, it achieves up to a **3.79x improvement in area efficiency** compared to conventional masking schemes while maintaining timing integrity across 65nm, 45nm, and 15nm technology nodes. This work further demonstrates that synthesis-level optimization can play a critical role in enhancing hardware security without incurring significant performance penalties. By integrating security considerations directly into the design flow, PoSyn provides a scalable and efficient alternative to traditional protection techniques. The experimental validation across multiple technology nodes confirms its adaptability and robustness. Additionally, the framework shows potential for integration into existing EDA toolchains, enabling widespread adoption in secure hardware design practices.

2.1.3 A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber[3] - IEEE Transactions on Circuits and Systems-II: Express Briefs.

Dejun Xu, Kai Wang, and Jing Tian (July, 2025)

This paper proposes a secure and efficient hardware implementation of **CRYSTALS-Kyber** designed to resist **Side-Channel Attacks (SCAs)**. The authors introduce a hardware-friendly modification of the **Fisher-Yates shuffle** algorithm and a novel **Random Permutation Generator (RPG)** architecture to randomize the execution order of critical operations. By shuffling sub-operation orders at all potential leakage points—including point-wise multiplication, modular reduction, and subtraction—the design significantly enhances security with minimal performance impact. Experimental results on FPGA verify the countermeasure's effectiveness through **Correlation Power Analysis (CPA)** and **Test Vector Leakage Assessment (TVLA)**, reporting only an **8.7% degradation** in hardware efficiency compared to unprotected versions.

2.1.4 Mutual Information Minimization for Side-Channel Attack Resistance via Optimal Noise Injection [4] - arXiv preprint, arXiv.

Jiheon Woo, Donggyun Ryu, Daewon Seo, Young-Sik Kim, Namyoon Lee, Yuval Cassuto, and Yongjune Kim (January, 2026)

This paper addresses the security threat posed by side-channel attacks (SCAs), which extract secret keys from physical leakages such as power consumption and electromagnetic emissions. The authors propose a novel defense framework based on optimal artificial noise injection, modeling the side-channel as a communication system and seeking to minimize the mutual information between sensitive data and physical observations. To optimize protection under practical power constraints, the study formulates and solves two convex optimization problems: minimizing the total mutual information and minimizing the maximum information leakage across all observations. The proposed method provides a mathematically rigorous approach to suppressing information leakage, making it particularly valuable for enhancing the security of resource-constrained hardware like Internet of Things (IoT) devices. Furthermore, the framework offers flexibility in balancing security and power overhead, enabling practical deployment while maintaining strong resistance against advanced and adaptive side-channel attack techniques.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing system, the scenario pertains to a physical attack targeting System-on-Chip (SoC) devices, where the attack's impact on these devices is assessed using voltage-based sensors within a VLSI simulation. This simulation aims to demonstrate the effectiveness of an attack detection mechanism in a hardware security context.

3.1.1 DETAILED EXISTING SYSTEM

In the current system, the focus is on evaluating the susceptibility of System-on-Chip (SoC) devices to physical attacks, with a particular emphasis on assessing the impact of such attacks through the utilization of voltage-based sensors within a VLSI (Very-Large-Scale Integration) simulation environment. This simulation serves as a controlled and realistic setting to emulate real-world scenarios where SoC devices might be vulnerable to various physical threats. The objective is to gauge the effectiveness of an attack detection mechanism designed to identify and mitigate potential security breaches in a hardware context.

Voltage-based sensors play a pivotal role in this assessment, as they are sensitive to fluctuations and anomalies in the electrical characteristics of the SoC. The simulation aims to replicate potential attack scenarios, such as voltage tampering or voltage glitching, which could compromise the normal functioning of the integrated circuits. By monitoring and analysing the voltage variations using these sensors, the system seeks to detect abnormal patterns indicative of a security breach.

The significance of this simulation lies in its ability to provide insights into the robustness of the attack detection mechanism under diverse attack scenarios. It allows for the identification of potential weaknesses in the SoC's security architecture and provides an opportunity to refine and strengthen the defence mechanisms against physical attacks. The real-time response of the detection mechanism in the simulated environment offers valuable data for enhancing the overall security posture of SoC devices. This further enables improved reliability, scalability, and adaptability of the system for deployment in real-world secure hardware applications.

In summary, the existing system employs a VLSI simulation framework coupled with voltage-based sensors to comprehensively evaluate the impact of physical attacks on SoC devices. Through this approach, the system aims to validate and optimize an attack detection mechanism, contributing to the ongoing efforts to fortify hardware security in the realm of integrated circuits and ensuring the resilience of SoC devices against potential threats.

3.2 PROBLEM STATEMENT

- In the existing system, only Physical attacks are considered for analysis
- Laser injection-based problems are discussed.
- Voltage modulation sensor, two-fold sensor validation is done.

3.3 PROPOSED SYSTEM

To overcome various clock issues in System on chip devices, multi tasking FPGA chips need to be tested and validated with physical attack detection modules. here an in-depth analysis module is created to test the FPGA devices with multiple clock tree synthesizers such as sweep generators, configurable clock synthesizers, Digital phase locked loop etc.

Advantages of proposed system

- To explore the idea of physical attack detection in SoC devices, application circuits need to be developed and tested.
- Multi-tasking clock synthesizers, Clock distributors need to be tested with multiple resources.

3.3.1 DETAILED PROPOSED

In addressing the intricate clock-related challenges prevalent in System-on-Chip (SoC) devices, a comprehensive testing and validation approach is employed, specifically focusing on multi-tasking FPGA chips. Recognizing the vulnerabilities associated with these devices, a key strategy involves the integration of physical attack detection modules to enhance security and resilience against potential threats. This approach ensures enhanced detection accuracy, improved system reliability, and robust protection.

A critical component of this testing framework is the in-depth analysis module, designed to thoroughly evaluate FPGA devices. This module employs various clock tree synthesizers, including sweep generators, configurable clock synthesizers, and Digital Phase-Locked Loops (DPLLs). The use of sweep generators allows for dynamic and adaptable clocking, while configurable clock synthesizers enhance flexibility, accommodating diverse operating conditions. DPLLs contribute to precision in clock synchronization, crucial for maintaining synchronous operations in complex SoC environments.

The multifaceted testing process involves subjecting the FPGA devices to diverse clock scenarios, simulating real-world conditions. This comprehensive evaluation aims to identify and rectify potential clock-related issues such as skew, jitter, and signal integrity challenges. The integration of multiple clock tree synthesizers ensures a thorough exploration of the FPGA's performance under varying clocking strategies, providing insights into its adaptability and reliability.

Simultaneously, the inclusion of physical attack detection modules addresses security concerns by actively monitoring and identifying potential threats. This proactive security approach aligns with the increasing need for robust defences against cyber-physical attacks, especially in the context of sensitive applications handled by SoC devices.

By combining in-depth clock analysis with physical attack detection, this testing and validation framework aims to fortify multi-tasking FPGA chips against a spectrum of challenges. The overarching goal is to ensure the reliability, security, and resilience of SoC devices, ultimately contributing to the advancement of trustworthy and high-performance embedded systems.

Furthermore, the proposed system can be extended by incorporating adaptive monitoring techniques that dynamically adjust detection thresholds based on real-time operating conditions. This enhancement would enable the system to respond more effectively to variations in workload, temperature, and environmental noise. Additionally, integrating on-chip diagnostic feedback mechanisms can facilitate faster fault localization and recovery. Such improvements would not only strengthen the overall robustness of the design but also support scalability for future high-performance and security-critical SoC applications, making the framework more versatile and implementation-ready.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 SOFTWARE SPECIFICATION

Simulation MODELSIM

ModelSim is a widely used simulation and verification tool for digital designs, particularly in hardware description languages (HDLs) like VHDL, Verilog, and System Verilog. It is primarily employed to simulate the behaviour of digital circuits at various stages of design, from RTL (Register Transfer Level) to gate-level simulation. ModelSim allows engineers to test and verify the functionality of their digital designs before physical implementation, ensuring that the circuit operates as intended and meets design specifications. One of its key features is its integrated debugging environment, which includes waveform viewers, breakpoints, and step-by-step execution, making it easier to trace and identify issues in the design.

The software supports both functional and timing simulations, enabling designers to not only verify logic correctness but also analyse how the circuit behaves under real-world timing constraints. ModelSim also integrates with various FPGA and ASIC design flows, making it a crucial part of the design and verification process for both small and large-scale digital systems. Its user-friendly interface, extensive library support, and ability to handle complex simulations make it a standard choice for hardware engineers aiming to ensure the accuracy and efficiency of their designs before committing to costly hardware fabrication.

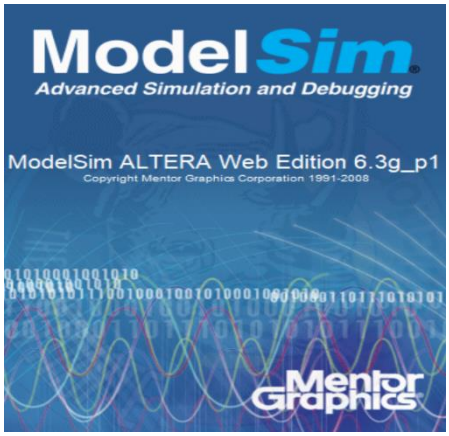

Fig. 4.1 ModelSim Altera 6.3g Simulation Environment for VLSI Design Verification

ModelSim offers a range of powerful features that make it a popular choice for simulating and verifying digital designs. Some of the key features include:

1. Multi-Language Support

- ModelSim supports multiple hardware description languages (HDLs) such as VHDL, Verilog, and System Verilog, making it a versatile tool for various design environments.

2. Waveform Viewer

- The integrated waveform viewer allows designers to visualize signal transitions and interactions over time. This makes it easier to debug and understand how the circuit operates.

3. Graphical User Interface (GUI)

- ModelSim provides a user-friendly GUI with various tools, such as source code viewers, variable explorers, and simulation controls, allowing for intuitive navigation and debugging.

4. Command-Line Interface (CLI)

- In addition to the GUI, ModelSim offers a robust command-line interface for scripting and automation, which is particularly useful for regression testing and complex simulations.

5. Fast and Efficient Simulation

- ModelSim is optimized for both functional simulation and timing simulation, ensuring fast performance even for large designs with complex logic and timing requirements.

6. Code Coverage

- It includes code coverage tools like statement, branch, and condition coverage, helping designers ensure that all parts of the design have been thoroughly tested and verified. This feature also aids in identifying untested scenarios and improving overall verification completeness and design reliability.

7. Testbench Support

- ModelSim allows the creation and integration of testbenches, which simulate inputs and monitor outputs for functional verification of the design. It supports self-checking testbenches for automated verification.

8. Debugging Tools

- The software includes powerful debugging features such as breakpoints, step-through simulation, and interactive variable inspection, which help engineers efficiently trace and fix design issues.

9. Assertions and Verification

- ModelSim supports System Verilog assertions (SVA) and provides mechanisms for formal verification, helping to catch bugs early and ensure the design behaves as expected under all conditions.

10. Design Navigation

- With hierarchical design support, ModelSim allows easy navigation through different levels of the design, from the top-level module to individual sub-modules and signals, streamlining the verification process.

11. FPGA and ASIC Design Flow Integration

- ModelSim integrates with leading FPGA and ASIC tools, such as Altera Quartus and Xilinx Vivado, allowing a seamless transition from simulation to synthesis and implementation.

12. Mixed-Signal Support

- ModelSim can simulate mixed-language environments, allowing for the integration of analog and digital components in a design, which is useful for mixed-signal applications. This capability enables designers to accurately model real-world system behavior by capturing interactions between analog and digital domains, facilitating comprehensive validation of complex systems such as communication interfaces, sensor-based designs, and embedded systems where both signal types coexist and influence overall performance.

13. Simulation Acceleration

- For large and complex designs, ModelSim offers features like simulation acceleration and incremental compilation, reducing simulation time and improving efficiency.

14. Scripting and Batch Processing

- The software supports TCL scripting for batch simulations and automating repetitive tasks, enhancing productivity for large verification projects.

15. Cross-Platform Availability

- ModelSim runs on multiple platforms, including Windows and Linux, making it accessible across different development environments.

The software specification presented in this chapter highlights the critical role of ModelSim in the design and verification of digital systems. Its comprehensive support for multiple hardware description languages, along with advanced simulation capabilities, ensures accurate functional and timing analysis of complex circuits. The availability of powerful debugging tools, waveform visualization, and code coverage features enables designers to thoroughly validate their designs, reducing the risk of errors before hardware implementation. Overall, ModelSim provides a reliable and efficient environment for achieving high-quality digital system design.

Furthermore, the seamless integration of ModelSim with FPGA and ASIC design flows greatly enhances its applicability in real-world engineering scenarios. Its advanced features, including mixed-signal support, scripting through TCL, simulation acceleration, and incremental compilation, significantly improve productivity, reduce development time, and support scalability for handling large and complex design projects. The availability of powerful debugging tools, waveform visualization, and code coverage features enables designers to thoroughly validate their designs. By enabling detailed behavioural analysis, precise timing verification, and automated validation through testbenches and assertions, the tool facilitates the development of highly robust, optimized, and error-free hardware systems. Consequently, ModelSim plays a vital role in strengthening the overall design and verification process, ensuring a high level of accuracy, efficiency, reliability, and performance in modern VLSI and embedded system development environments.

CHAPTER 5

METHODOLOGY

5.1 MODULE DESCRIPTION

MODULE 1: REFERENCE CLOCK GENERATOR

It is developed using a global clock (synchronizes whole for the whole chip), and global reset signal (forces everything back to a known starting state), which act as the primary timing and initialization controls for the entire chip. A 20-bit clock divider (different speeds can be generated for testing) is designed to generate multiple clock frequencies required for testing and analysis.

MODULE 2: TEST CIRCUIT GENERATOR

From the reference clock generator, a sweep generator (slowly changes clock speed up and down), clock synthesizer (creates different clock frequencies from one source) and digital PLL (keeps clocks stable and aligned) are implemented. These modules enable controlled clock variation, frequency synthesis, and clock stabilization. The architecture is configurable and incorporates low-power design techniques.

MODULE 3: DESIGN OF ATTACK PATTERN GENERATOR

A dedicated module is designed to generate attack patterns, including side-channel attacks, FPGA replay attacks, and corruption attacks. These digital circuits emulate real-world attack scenarios to evaluate vulnerabilities in SoCs, FPGAs, and Embedded Systems.

MODULE 4: DESIGN OF DYNAMIC PATTERN MATCHING ALGORITHM

A dynamic pattern-matching algorithm [watches behavior (power, timing, signal patterns) and to compares with known bad patterns**]** is implemented to monitor power, timing, signal behavior, and to compare observed patterns with known attack signatures for detection and validation. This algorithm operates in real time, continuously analyzing variations in system parameters to detect anomalies that may indicate potential attacks or faults. By utilizing adaptive thresholds and pattern learning techniques, it enhances detection accuracy while minimizing false positives, thereby ensuring reliable and efficient monitoring of hardware security in complex FPGA-based systems.

MODULE 5 : INTEGRATION

A finite state machine (FSM) (controller that ensures correct execution order) is designed to control execution flow and coordinate all submodules. The final system integrates all modules to operate cohesively without functional conflicts.

5.2 DESIGN DESCRIPTION

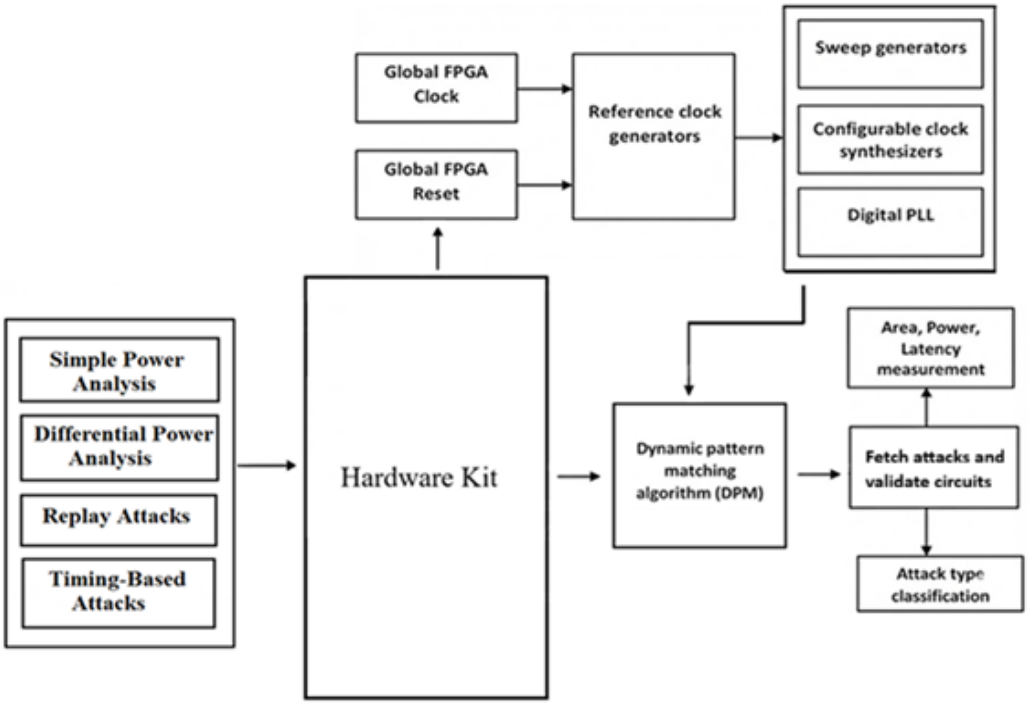

Fig. 5.1 Block Diagram of FPGA-Based Side-Channel Attack Detection and System

5.3 IMPLEMENTATION SUMMARY

In your scenario, it seems that you are conducting a simulation to assess how voltage-based sensors can detect the impact of physical attacks on SoC devices. This simulation would involve modeling the SoC device, the voltage-based sensors, and various types of physical attacks to evaluate the effectiveness of the attack detector. In addition to the initial setup, the implementation also considers realistic operating conditions by incorporating variations in supply voltage, temperature fluctuations, and process variations that commonly affect SoC performance. Advanced monitoring techniques are integrated to capture transient, the simulation framework is designed to emulate both normal and adversarial environments, enabling a comprehensive evaluation of system behaviour under stress conditions.

Some possible steps in such a simulation could include:

- Developing a detailed model of the SoC device, including its components and power consumption characteristics.
- Implementing voltage-based sensors to monitor the SoC device's voltage levels.
- Simulating different types of physical attacks, such as voltage spikes, signal interference, or attempts to tamper with the hardware.
- Analyzing the sensor data to detect anomalies or deviations from expected behavior that may indicate an attack.
- Assessing the performance of the attack detector in terms of its ability to accurately detect and respond to attacks while minimizing false positives.
- This type of simulation is essential for evaluating the robustness and security of SoC devices and can help in designing more secure hardware systems.
- Incorporating adaptive threshold mechanisms to dynamically adjust detection sensitivity based on operating conditions.
- Integrating machine learning-based classifiers to improve anomaly detection and reduce false positives.
- Extending the simulation to include additional attack models such as electromagnetic interference and fault injection.
- Evaluating system performance under different clock frequencies and workload conditions.
- Implementing real-time alert mechanisms for faster response to detected attacks.
- Comparing detection accuracy across multiple sensor configurations for optimal design selection.

Overall, the implementation demonstrates a systematic approach to evaluating the effectiveness of voltage-based sensors in detecting physical attacks on SoC devices. By combining detailed modelling, realistic simulation scenarios, and comprehensive analysis techniques, the framework ensures accurate detection and reliable system performance. The results highlight the importance of proactive monitoring and robust detection mechanisms in enhancing hardware security. This approach not only improves the resilience of SoC systems but also provides a strong foundation for developing advanced and scalable security solutions in future embedded system designs.

CHAPTER 6

SOURCE CODE

6.1 BUFFER MODULE

```
Library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;


entity BUFF is
port(clk,clr,upcnt: in std_logic;
carry,barrow : out std_logic);
end BUFF;


architecture behave of BUFF is
signal upcounter , downcounter : std_logic_vector(7 downto 0);

begin

lpfilter: process(clk,clr,upcnt)
begin
   if clr='1' then
      upcounter<="00000000";
      downcounter<="11111111";
      elsif rising_edge(clk) then
         if upcnt = '1' then
         upcounter  <=   upcounter+1;
         else
         downcounter<=downcounter-1;
      end if;
end if;
end process lpfilter;

carry   <=   upcounter(7);
barrow  <=   downcounter(7);

end behave;
```

6.2 REFERENCE SIGNAL GENERATOR MODULE

```
Library ieee;

use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
```

```
use ieee.std_logic_arith.all;

entity ref_sig_gen is
port(clk,clr: in std_logic;
    sel: in std_logic_vector(7 downto 0);
    ref_clk : out std_logic);
end ref_sig_gen;

architecture behave of  ref_sig_gen is
signal clkcounter_i,clkcounter : std_logic_vector(19 downto 0);
signal asig: std_logic;
begin

-- Clock divider
Clkdiv : process(clk,clr)
begin
    if clr='1' then
        clkcounter_i<="00000000000000000000";
        elsif rising_edge(clk) then
        clkcounter_i<= clkcounter_i +1;
end if;
end process Clkdiv;


clkcounter <= clkcounter_i;

-- Data Selector
data_selector: process(sel,clr,clk)
begin
if clr ='1' then
    ref_clk <='0';
    asig<='1';
    elsif rising_edge(clk) then

     if sel="00000001" then
       ref_clk      <=       asig and clkcounter(0);


      else if sel="00000010" then
       ref_clk      <=       asig and clkcounter(1);

      else if sel="00000011" then
       ref_clk      <=       asig and clkcounter(2);


      else if sel="00000100" then
       ref_clk      <=       asig and clkcounter(3);
```

```
else if sel="00000101" then
  ref_clk     <=      asig and clkcounter(4);


 else if sel="00000110" then
  ref_clk     <=      asig and clkcounter(5);


 else if sel="00000111" then
  ref_clk     <=      asig and clkcounter(6);


 else if sel="00001000" then
  ref_clk     <=      asig and clkcounter(7);

 else if sel="00001001" then
  ref_clk     <=      asig and clkcounter(8);


 else if sel="00001010" then
  ref_clk     <=      asig and clkcounter(9);


 else if sel="00001011" then
  ref_clk     <=      asig and clkcounter(10);


 else if sel="00001100" then
  ref_clk     <=      asig and clkcounter(11);


 else if sel="00001101" then
  ref_clk     <=      asig and clkcounter(12);


 else if sel="00001110" then
  ref_clk     <=      asig and clkcounter(13);


 else if sel="00001111" then
  ref_clk     <=      asig and clkcounter(14);


 else if sel="00010000" then
  ref_clk     <=      asig and clkcounter(15);


 else if sel="00010001" then
  ref_clk     <=      asig and clkcounter(16);
```

```
      else if sel="00010010" then
       ref_clk     <=       asig and clkcounter(17);


      else if sel="00010011" then
       ref_clk     <=       asig and clkcounter(18);


      else
       ref_clk     <=       asig and clkcounter(19);
       end if;

end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end if;
end process data_selector;
end behave;
```

6.3 CLOCK SYNTHESIZER MODULE

```
library ieee;

use ieee.std_logic_1164.all;

entity Clock_Synth is
port(clk,clr,a,b,c,d: in std_logic;
    ykk:in std_logic_vector(6 downto 0);
y,yss: out std_logic_vector(6 downto 0));
end Clock_Synth;

architecture behave of Clock_Synth is
```

```
signal p1,p2,p3,p4,s1,s2,q0,q1,q2,k1,k2,k3,y1: std_logic;

begin
t1:process(clk,k1,ykk)
begin
if clr='1' or ykk(6)='1' then
q0<='0';
elsif rising_edge(clk) then
q0<=k1;
end if;
end process t1;


t2:process(clk,k2,ykk)
begin
if clr='1' or ykk(6)='1'then
q1<='0';
elsif rising_edge(clk) then
q1<=k2;
end if;
end process t2;

t3:process(clk,k3,ykk)
begin
if clr='1' or ykk(6)='1'then
q2<='0';
elsif rising_edge(clk) then
q2<=k3;
end if;
end process t3;
-- Gate functions
s1      <= q1 nor a;
s2      <= (not d ) and q2;

p1      <= s1 or s2;
p2      <=s2 or c;
p3      <= p1 nand p2;
p4      <=s1 nor p3;

y1      <= not p4;
k1<= (not d) nor p4;
k2<=b nor s1;
k3<=p4;

y(0)<=P1 or ykk(0);
y(1)<=p2 or ykk(1);
y(2)<=p3 or ykk(2);
y(3)<=p4 or ykk(3);
y(4)<=k1 or ykk(4);
```

```
y(5)<=k2 or ykk(5);
y(6)<=k3 or ykk(6);

yss(0)<=P1;
yss(1)<=p2;
yss(2)<=p3;
yss(3)<=p4;
yss(4)<=k1;
yss(5)<=k2;
yss(6)<=k3;

end behave;
```

6.4 BENCHMARK CIRCUIT 1 MODULE

```
library ieee;
use ieee.std_logic_1164.all;

entity BENCHMARK_CKT1 is
port(clk,clr,aa,ba,ca,da: in std_logic;
ya: out std_logic);
end BENCHMARK_CKT1;

architecture behave of BENCHMARK_CKT1 is
signal p1a,p2a,p3a,p4a,s1a,s2a,q0a,q1a,q2a,k1a,k2a,k3a: std_logic;

begin
t1a:process(clk,k1a)
begin
if clr='1' then
q0a<='0';
elsif rising_edge(clk) then
q0a<=k1a;
end if;
end process t1a;


t2a:process(clk,k2a)
begin
if clr='1' then
q1a<='0';
elsif rising_edge(clk) then
q1a<=k2a;
end if;
end process t2a;

t3a:process(clk,k3a)
begin
if clr='1' then
```

```
q2a<='0';
elsif rising_edge(clk) then
q2a<=k3a;
end if;
end process t3a;
-- Gate functions
s1a     <= q1a nor aa;
s2a     <= (not da ) and q2a;

p1a     <= s1a or s2a;
p2a     <=s2a or ca;
p3a     <= p1a nand p2a;
p4a     <=s1a nor p3a;

ya      <= not p4a;
k1a<= (not da) nor p4a;
k2a<=ba nor s1a;
k3a<=p4a;

end behave;
```

6.5 BENCHMARK CIRCUIT 2 MODULE

```
library ieee;
use ieee.std_logic_1164.all;

entity BENCHMARK_CKT2 is
port(clk,clr,a,b,c,d: in std_logic;
yu: out std_logic);
end BENCHMARK_CKT2;

architecture behave of BENCHMARK_CKT2 is
signal p1,p2,p3,p4,s1,s2,q0,q1,q2,k1,k2,k3: std_logic;

begin
t1:process(clk,k1)
begin
if clr='1' then
q0<='0';
elsif rising_edge(clk) then
q0<=k1;
end if;
end process t1;

t2:process(clk,k2)
begin
if clr='1' then
q1<='0';
elsif rising_edge(clk) then
```

```
q1<=k2;
end if;
end process t2;

t3:process(clk,k3)
begin
if clr='1' then
q2<='0';
elsif rising_edge(clk) then
q2<=k3;
end if;
end process t3;

-- Gate functions
s1      <= q1 nor a;
s2      <= (not d ) and q2;

p1      <= s1 or s2;
p2      <=s2 or c;
p3      <= p1 nand p2;
p4      <=s1 nor p3;

yu      <= not p4;
k1<= (not d) nor p4;
k2<=b nor s1;
k3<=p4;
end behave;
```

6.6 ATTACK PATTERN GENERATOR MODULE

```
Library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity Attack_gen is
   port(clk,clr: in std_logic;
   config : in std_logic_vector(3 downto 0);
   Attack_Insert: out std_logic_vector(6 downto 0));
end Attack_gen;


architecture behave of Attack_gen is
SIGNAL Attack_Insert1,Attack_Insert2,Attack_Insert3:  std_logic_vector(6 downto 0);
begin

--Information leaking
--Information leaking: an FPGA vendor?s confidential data needed
--for the fabrication of the FPGA chips may be mishandled or leaked
```

```
--to parties (e.g., another competing FPGA vendor) that should not
--have access to such data (Pw controlled)

Unauthorized_Probe:process(clk,clr,config(0))
begin
   if clr='1' or config(0)='0'then
      Attack_Insert1<="0000000";
      elsif rising_edge(clk) then
      Attack_Insert1<="1000000";
   end if;
end process Unauthorized_Probe;

--Reverse engineering:
--IP core vendors expect their IPs cannot be reverse engineered
--by untrusted parties (Hacking of Data- Insertion detection)

Data_Insert:process(clk,clr)
begin
   if clr='1' or config(1)='0' then
      Attack_Insert2<="1011101";
      elsif rising_edge(clk) then
      Attack_Insert2<=Attack_Insert2+1; -- Fetch Wrong Data into the Logic
   end if;
end process Data_Insert;


--FPGA replay attack:
--an adversary downgrades an FPGA-based system
--to the previous version of FPGA chips with known vulnerabilities
--and explores such vulnerabilities. (Forbidden_Resets)

Forbidden_Resets:process(clk,clr,config(2))
begin
   if clr='1' or config(2)='0' then
      Attack_Insert3<="0000000";
      elsif rising_edge(clk) then
      Attack_Insert3(0)<=Attack_Insert2(2) xor Attack_Insert2(1) xor Attack_Insert3(3); --
Fetch reset loop
      Attack_Insert3(1)<=Attack_Insert2(4) xor Attack_Insert2(5) xor Attack_Insert3(4); --
Fetch reset loop
      Attack_Insert3(2)<=Attack_Insert2(6) xor Attack_Insert2(1) xor Attack_Insert3(1); --
Fetch reset loop
      Attack_Insert3(3)<=Attack_Insert2(2) xor Attack_Insert2(4) xor Attack_Insert3(2); --
Fetch reset loop
      Attack_Insert3(4)<=Attack_Insert2(3) xor Attack_Insert2(1) xor Attack_Insert3(5); --
Fetch reset loop
     Attack_Insert3(5)<=Attack_Insert2(2) xor Attack_Insert2(6) xor Attack_Insert3(6); --
Fetch reset loop
```

```
      Attack_Insert3(6)<=Attack_Insert2(2) xor Attack_Insert2(2) xor Attack_Insert3(0); --
Fetch reset loop
   end if;
end process Forbidden_Resets;

AttackRouter:process(clk,clr,config)
begin
if clr='1' then
Attack_Insert<="0000000";
elsif falling_edge(clk) then
   case config is
      when "0001"=>Attack_Insert<=Attack_Insert1;
      when "0010"=>Attack_Insert<=Attack_Insert2;
      when "0100"=>Attack_Insert<=Attack_Insert3;
      when others =>Attack_Insert<="0000000";
   end case;
end if;
end process AttackRouter;

end behave;
```

6.7 DELAY DETECTOR MODULE

```
Library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity delay_detector is
   port(fin,fref : in std_logic;
      lock: out std_logic;
      up,down : out std_logic);
end delay_detector;


architecture behave of delay_detector is
   signal clry: std_logic;
   signal aout,bout : std_logic;
   signal aout_bar,bout_bar : std_logic;
   signal vcc: std_logic;

begin

--Get Input Frequency
vcc<='1';
clry<= aout nand bout;

a: process(fin,clry)
begin
```

```
if clry='1' then
    aout<='0';

elsif rising_edge(fin) then
    aout<=vcc;
    aout_bar<= not vcc;
end if;
end process a;


-- Get reference frequency

b: process(fref,clry)
begin
if clry='1' then
  bout<='0';
elsif rising_edge(fref) then
  bout <= vcc;
  bout_bar<= not vcc;
end if;
end process b;

Lock <= aout_bar and bout_bar;

up    <= aout_bar;
down  <= bout_bar;

end behave;
```

6.8 LOGIC ATTACK DETECTOR MODULE

```
Library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity Logic_Attack_Detector is
   port(clk,clr : in std_logic;
       a,b,c,d: in std_logic;
       config: in std_logic_vector(3 downto 0);
       y: out std_logic_vector(6 downto 0));
end Logic_Attack_Detector;

architecture behave of Logic_Attack_Detector is
  COMPONENT Attack_gen is
  port(clk,clr: in std_logic;
  config : in std_logic_vector(3 downto 0);
   Attack_Insert: out std_logic_vector(6 downto 0));
  end COMPONENT;
```

```
  COMPONENT Clock_Synth is
  port(clk,clr,a,b,c,d: in std_logic;
     ykk:in std_logic_vector(6 downto 0);
  y,yss: out std_logic_vector(6 downto 0));
  end COMPONENT;

  COMPONENT BENCHMARK_CKT1 is
  port(clk,clr,aa,ba,ca,da: in std_logic;
  ya: out std_logic);
  end COMPONENT;

  COMPONENT BENCHMARK_CKT2 is
  port(clk,clr,a,b,c,d: in std_logic;
  yu: out std_logic);
  end COMPONENT;

  COMPONENT delay_detector is
   port(fin,fref : in std_logic;
      lock: out std_logic;
      up,down : out std_logic);
  end COMPONENT;

signal    ai:std_logic_vector(6 downto 0);
signal    yk,ys,ysk:std_logic_vector(6 downto 0);
signal    sg,tc1,tc2,tc3,tc4,tc5,tc6: std_logic;
signal    cnt: std_logic_vector(3 downto 0);
signal    cntt: integer;
signal    VCheck: string(1 to 10);
signal    sk1,sk2,sk3,sk4,sk5,sk6,skk: std_logic_vector(3 downto 0);
signal    yo2,yo3,locka,upa,downa,dnode:  std_logic;
begin
AttackGen   : Attack_gen port map(clk,clr,config,ai);
TestCkt     : Clock_Synth port map(clk,yk(6),a,b,c,d,yk,ys,ysk);
TestCkt2    : BENCHMARK_CKT1 port map (clk,dnode,a,b,c,d,yo2);
TestCkt3    : BENCHMARK_CKT2 port map (clk,dnode,ys(6),b,ys(5),d,yo3);
Latency_Attack_detector    : delay_detector port map(yo2,yo3,locka,upa,downa);

-- Fetch Attack
tc1<=( config(0)) and (not config(1)) and (not config(2)) and (not config(3));
tc2<=(not config(0)) and ( config(1)) and (not config(2)) and (not config(3));
tc3<=(not config(0)) and (not config(1)) and ( config(2)) and (not config(3));
tc4<=(not config(0)) and (not config(1)) and (not config(2)) and ( config(3));
tc5<=(config(0)) and (config(1)) and (not config(2)) and ( not config(3));
tc6<=(not config(0)) and ( config(1)) and ( config(2)) and (not config(3));

FA:process(clk,clr)
begin
   if clr='1' then
```

```
       yk<="0000000";
     elsif rising_edge(clk) then
        yk<=ai;
     end if;
end process FA;

-- Test Validator
TV: process(clk,clr)
begin
   if clr='1' then
      cnt<="0000";
      elsif falling_edge(clk) then
         cnt<=cnt+1;
         sg<='1';
         if cnt>"0111" then
            sg<='0';
         end if;
      end if;
end process TV;


AttackDetector: process(clk,clr,ysk,tc1)
begin
   if clr='1' or tc1='0' then
      cntt<=0;
      sk1<="0000";
      elsif falling_edge(clk) then
            cntt<=cntt+1;
            if cntt>10000 then
               sk1<="0001";
            end if;


      end if;
end process AttackDetector;

pp:process(clk,clr,tc2)
begin
   if clr='1' or tc2='0'then
      sk2<="0000";
      elsif rising_edge(clk) then
       if (ys and (not ysk)) > "0000000" then
            sk2<="0010";
         end if;
end if;
end process pp;

-- The below process acts as a positive edge triggered flip flop which detect the
-- occurence of attack insert triggered by the majority check AND-DECODER and reflect
```

```
-- the changes in sk3
pp2:process(clk,clr,tc3)
begin
   if clr='1' or tc3='0' then
      sk3<="0000";
      elsif rising_edge(clk) then
            sk3<="0100";
    end if;
end process pp2;
-- The below process acts as a positive edge triggered flip flop which detect the
-- occurence of attack insert triggered by the majority check AND-DECODER and reflect
-- the changes in sk4
pp3:process(clk,clr,tc4)
begin
   if clr='1' or tc4='0' then
      sk4<="0000";
      elsif rising_edge(clk) then
            sk4<="0101";
    end if;
end process pp3;

-- The below process acts as a positive edge triggered flip flop which detect the
-- occurence of attack insert triggered by the majority check AND-DECODER and reflect
-- the changes in sk5

pp55:process(clk,clr,tc5)
begin
   if clr='1' or tc5='0' then
      sk5<="0000";
      elsif rising_edge(clk) then
            sk5<="0110";
    end if;
end process pp55;
-- The below process acts as a positive edge triggered flip flop which detect the
-- occurence of attack insert triggered by the majority check AND-DECODER and reflect
-- the changes in sk6
pp66:process(clk,clr,tc6)
begin
   if clr='1' or tc6='0' then
      sk6<="0000";
      elsif rising_edge(clk) then
            sk6<="0111";
    end if;
end process pp66;
-- The Encoder below encodes the attacks inserted and make the LUT selection
appropriatey
-- for making the notification

skk<=sk1 or sk2 or sk3 or sk4 or sk5 or sk6;
```

```
-- The clocked LUT (Look up Table) displays the attack names
AttackNotifier:process(clk,clr,skk)
begin
   if clr='1' then
      VCheck<="Normal****";
      elsif falling_edge(clk) then
         case skk is
            when "0001"=>VCheck<="ProbAttack";
            when "0010"=>VCheck<="CorrAttack";
            when "0100"=>VCheck<="forbidAttk";
            when "0101"=>VCheck<="ReplayATK*";
            when "0110"=>VCheck<="DeadNode**";
            when "0111"=>VCheck<="Side_Chan*";
            when others=> VCheck<="Normal****";
         end case;
      end if;
   end process AttackNotifier;
   y<=ysk;
   dnode<=skk(1) and skk(2);
end behave;
```

6.9 TESTBENCH MODULE

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_unsigned.all;


ENTITY TestBench_LAD IS

END TestBench_LAD;

ARCHITECTURE behavior OF TestBench_LAD IS

COMPONENT Logic_Attack_Detector is
   port(clk,clr : in std_logic;
      a,b,c,d: in std_logic;
      config: in std_logic_vector(3 downto 0);
      y: out std_logic_vector(6 downto 0));
end COMPONENT;


  signal clk : std_logic := '0';
 signal clr : std_logic := '1';
  signal a,b,c,d:std_logic := '0';
  signal y:  std_logic_vector(6 downto 0);
SIGNAL    config: std_logic_vector(3 downto 0);
  constant clk_period  : time := 100 ps;
    constant clk_period1  : time := 1000 ps;
```

```
BEGIN

TB_LAD: Logic_Attack_Detector PORT MAP (clk,clr,a,b,c,d,config,y);

  clk_process :process

  begin

       clk <= '0';
       wait for clk_period/2;  --for 0.5 ns signal is '0'.
       clk <= '1';
       wait for clk_period/2;  --for next 0.5 ns signal is '1'.

  end process;


  -- Stimulus process

  stim_proc: process

  begin

       wait for 700 ps;
       clr <='1';
       wait for 300 ps;
       clr <='0';
       wait;

  end process;


  stim_proc_2: process

  begin

       wait for 50 ps;
       a <='0';b<='0';c<='0';d<='0';
       wait for 100 ps;
       a <='0';b<='0';c<='0';d<='1';
       wait for 100 ps;
       a <='0';b<='0';c<='1';d<='0';
       wait for 100 ps;
       a <='0';b<='0';c<='1';d<='1';
       wait for 100 ps;
       a <='0';b<='1';c<='0';d<='0';
       wait for 100 ps;
       a <='0';b<='1';c<='0';d<='1';
       wait for 100 ps;
```

```
        a <='0';b<='1';c<='1';d<='0';
        wait for 100 ps;
        a <='0';b<='1';c<='1';d<='1';
        wait for 100 ps;
        a <='1';b<='0';c<='0';d<='0';
        wait for 100 ps;
        a <='1';b<='0';c<='0';d<='1';
        wait for 100 ps;
        a <='1';b<='1';c<='0';d<='0';
        wait for clk_period/2;

    end process stim_proc_2;

    stim_procGG: process

     begin

        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="0001";
        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="0010";
        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="0100";
        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="1000";
        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="0011";
        wait for 5000 ps;
        config <="0000";
        wait for 5000 ps;
        config <="0110";
        wait for 5000 ps;
        config <="0000";

     end process stim_procGG;

    END;
```

CHAPTER 7

TESTING AND VALIDATION

Testing procedures for VLSI (Very Large-Scale Integration) code involve a series of steps to ensure that the designed integrated circuits (ICs) meet the required specifications and function correctly. Here's a detailed overview of testing procedures for VLSI code:

7.1 DESIGN VERIFICATION

Pre-simulation checks:

Syntax errors: Ensure that the HDL code adheres to the syntax rules of the language being used (e.g., Verilog, VHDL). Common syntax errors include missing semicolons, incorrect module port declarations, and misspelled keywords.

Logical errors: Verify the logical correctness of the design by reviewing the RTL (Register Transfer Level) code. Check for unintended behavior, incorrect data paths, and functional inconsistencies.

Connectivity issues: Ensure that all signals are properly connected within the design hierarchy. Verify that module instances are instantiated correctly and that signal names match between the module interface and the instantiated connections.

Functional simulation:

HDL testbenches: Develop test bench code written in the same HDL as the design to verify its functionality. Testbenches generate stimulus (input vectors) to apply to the design and monitor the responses (output signals).

Operational scenarios: Design test cases to cover various operational scenarios, including normal operation, boundary conditions, error conditions, and corner cases. Stimulate the design with input vectors that exercise different paths and functionalities.

Code coverage analysis: Code coverage metrics: Utilize code coverage analysis tools to measure the extent to which the design code has been exercised by the testbench. Common code coverage metrics include statement coverage, branch coverage, and condition coverage.

Coverage goals: Set coverage goals based on project requirements and industry standards. This helps in identifying untested scenarios, improving design reliability, and ensuring robustness, aim to achieve high coverage levels to ensure thorough testing of the design.

Assertion-based verification:

Assertions: Insert assertion statements within the HDL code to specify expected behavior and check for violations of design properties. Assertions serve as self-checking mechanisms to detect errors during simulation.

Property checking: Define properties that capture the intended behavior of the design, such as signal values, timing constraints, and protocol compliance. Assertions evaluate these properties during simulation and report any violations.

These steps collectively form a comprehensive verification process aimed at ensuring the correctness and reliability of the VLSI design. By following these procedures rigorously, designers can identify and rectify issues early in the design cycle, ultimately leading to a more robust and dependable final product.

7.2 TIMING VERIFICATION

Static Timing Analysis (STA):

Setup Time: STA ensures that the data signal is stable before the clock edge arrives. It checks if the flip-flop input signals meet the required setup time before the active clock edge.

Hold Time: STA verifies that the data signal remains stable after the clock edge has arrived. It checks if the flip-flop input signals meet the required hold time after the active clock edge.

Maximum Clock Frequency: STA determines the maximum frequency at which the design can operate while meeting timing constraints. It identifies the critical path in the design and calculates the clock period based on the delay along the critical path.

Timing Violations: STA identifies any violations of setup time, hold time, or maximum clock frequency constraints. Violations can lead to functional errors or performance degradation and need to be addressed through design optimization techniques such as pipeline insertion, logic restructuring, or timing budget allocation, resolving timing violations improves overall circuit reliability and ensures stable operation.

Clock Domain Crossing (CDC) Analysis:

Proper Synchronization: CDC analysis ensures that signals crossing between different clock domains are properly synchronized to avoid metastability issues and data loss. It

identifies asynchronous crossings and determines the required synchronization techniques, such as double or triple flop synchronizers, gray coding, or handshake protocols.

Metastability Mitigation: CDC analysis assesses the impact of metastability on the design and evaluates the effectiveness of the chosen synchronization methods in mitigating metastability. Techniques such as metastability-aware simulation and timing margin analysis may be employed to validate the robustness of the synchronization approach.

Power Analysis:

Power Consumption Estimation: Power analysis estimates the dynamic and static power consumption of the design components under various operating conditions. It considers factors such as switching activity, capacitive load, and leakage current to calculate power dissipation.

Power Distribution Analysis: Power analysis evaluates the distribution of power within the design to identify hotspots and ensure compliance with power budget constraints. Techniques such as power grid analysis and IR drop analysis are used to assess voltage drops and ensure stable power supply across the chip.

7.3 FUNCTIONAL VERIFICATION

Gate-Level Simulation:

Functionality Verification: Gate-level simulations verify the correct functionality of the design after synthesis and optimization. The synthesized netlist, which consists of gates and flip-flops, is simulated to ensure that it behaves as intended.

Timing Behavior Verification: Gate-level simulations also evaluate the timing behavior of the design, considering gate delays, interconnect delays, and clock skew. This ensures that the design meets timing constraints specified during synthesis and optimization. This helps in identifying potential timing violations early and ensures reliable circuit operation under real-time conditions and varying operating environments.

Signal Integrity Checks: Gate-level simulations help identify signal integrity issues such as glitches, race conditions, and timing violations that may arise due to the physical implementation of the design. This enables designers to detect potential failures early and implement corrective measures to ensure stable and reliable circuit operation.

Design for Testability (DFT):

Scan Chains: DFT techniques like scan chains enable efficient testing of integrated circuits by serially shifting in test patterns and capturing output responses. Scan chains facilitate scan-based testing, which is widely used in manufacturing testing.

Built-In Self-Test (BIST) Circuits: BIST circuits are embedded within the design to perform self-testing without requiring external test equipment. BIST circuits generate test patterns, apply them to the circuit under test, and analyze the responses to detect faults.

Boundary Scan (JTAG): Boundary scan, standardized by the Joint Test Action Group (JTAG), provides a means for testing and debugging the interconnections between integrated circuits on a printed circuit board (PCB). JTAG enables accessing and controlling individual pins of the ICs for testing purposes.

Fault Simulation:

Stuck-At Faults: Fault simulation involves injecting various fault models into the gate-level netlist to evaluate the design's robustness against manufacturing defects. Stuck-at faults simulate permanent faults where a signal is stuck at a constant logic value (0 or 1).

Bridging Faults: Bridging faults simulate shorts between two or more signal lines, potentially causing unintended connections and logical errors. Fault simulation assesses the design's ability to detect and tolerate bridging faults.

Other Fault Models: Additional fault models such as transition faults, delay faults, and coupling faults may also be simulated to evaluate the design's fault coverage and ensure comprehensive testing.

By conducting gate-level simulations, implementing DFT features, and performing fault simulations, designers can ensure that the VLSI design is thoroughly tested for functionality, timing behavior, testability, and robustness against manufacturing defects, thereby improving its reliability and manufacturability.

7.4 PHYSICAL VERIFICATION

Design Rule Checking (DRC) is a crucial step in the VLSI design process, ensuring that the layout of the integrated circuit (IC) complies with the manufacturing constraints and design rules provided by the foundry. DRC is performed after the layout design is completed and before the fabrication process begins. During DRC, specialized software tools analyze

the layout design files to detect any violations of the specified design rules. These design rules encompass various aspects of the layout, including minimum feature sizes, spacing between features, width of interconnects, and alignment of components. By meticulously examining the layout against these rules, DRC ensures that the final IC design is manufacturable and meets the quality standards required for reliable operation. Common violations identified during DRC include short circuits, narrow spacing violations, overlapping features, and incorrect layer assignments. Designers must meticulously address these violations by modifying the layout design to comply with the specified rules. Failure to resolve DRC violations can result in manufacturing defects, yield loss, and ultimately, compromised performance or functionality of the fabricated IC. Therefore, DRC plays a critical role in guaranteeing the integrity and manufacturability of VLSI designs, contributing to the successful realization of high-quality integrated circuits

Layout vs. Schematic (LVS) verification is an essential step in the VLSI design flow, ensuring the consistency and correctness between the layout netlist and the schematic netlist of an integrated circuit (IC). LVS is performed after the layout design is completed and before fabrication to ensure that the physical layout accurately reflects the intended circuit design.

During LVS verification, specialized software tools compare the connectivity and topology of the layout netlist extracted from the layout design with the schematic netlist generated from the original circuit schematic. The goal is to identify any discrepancies or inconsistencies between the two representations of the circuit.

Key aspects of LVS verification include:

Connectivity Verification: The software tools analyze the connectivity of the components, nodes, and interconnects in both the layout and schematic netlists. It checks that all connections specified in the schematic are correctly realized in the layout and that there are no missing or extraneous connections.

Device Matching: LVS verifies that the physical devices (transistors, resistors, capacitors, etc.) in the layout correspond to the components in the schematic and are placed correctly according to the design specifications. This ensures functional equivalence between schematic and layout, minimizing fabrication errors and improving overall accuracy, reliability, and manufacturability of the integrated circuit design.

Netlist Comparison: The software performs a detailed comparison of the netlists, identifying any differences in node names, connectivity, or device parameters between the layout and schematic representations of the circuit.

Error Detection and Resolution: If discrepancies are found during LVS verification, the software generates error reports indicating the nature and location of the inconsistencies. Designers must then review and resolve these errors by modifying either the layout or schematic to ensure alignment between the two representations.

Validation and Sign-Off: Once all errors are resolved, a successful LVS verification indicates that the layout accurately implements the circuit design as specified in the schematic. This validation is a critical milestone before proceeding to the fabrication stage.

By performing LVS verification, designers ensure the integrity and correctness of the layout design, minimizing the risk of manufacturing defects and ensuring the functionality and performance of the fabricated IC. LVS is an indispensable part of the VLSI design flow, contributing to the successful realization of complex integrated circuits with high reliability and performance.

Electrical rule checking (ERC): Analyzing the layout for potential electrical issues such as short circuits, open circuits, and antenna violations is a critical step in the VLSI design process to ensure the integrity and reliability of the integrated circuit (IC). Here's an analysis of each of these potential issues:

1. **Short Circuits**:
    - **Definition**: A short circuit occurs when two or more conductive paths unintentionally connect, creating a low-resistance path between nodes with different potentials. This can result in excessive current flow, overheating, potential component damage, and malfunction of the overall circuit system.
    - **Analysis**: Layout verification tools perform a thorough check to identify any overlapping or closely spaced conductive elements that could result in a short circuit. Shorts can occur between metal layers, between metal and diffusion layers, or between any other adjacent layers.
    - **Resolution**: Designers resolve short circuits by modifying the layout to increase spacing between the offending conductive elements or by inserting isolation structures such as guard rings or dummy fill.

2. **Open Circuits**:

   - **Definition**: An open circuit occurs when there is an unintended break or discontinuity in a conductive path, preventing the flow of current between nodes.
   - **Analysis**: Layout verification tools inspect the layout for any gaps, missing connections, or discontinuities in the conductive paths. Open circuits can occur due to incomplete metal routing, missing vias, or misalignment between layers.
   - **Resolution**: Designers address open circuits by adding missing routing segments, ensuring proper alignment of metal layers, or inserting additional vias to establish connectivity between layers.

3. **Antenna Violations**:

   - **Definition**: Antenna violations occur when unconnected metal nodes are exposed to plasma during the manufacturing process, leading to the accumulation of charge and potential damage to the device.
   - **Analysis**: Layout verification tools identify unconnected metal nodes that could act as antennas during the fabrication process. Antenna violations are particularly common in designs with floating gates or nodes.
   - **Resolution**: Designers mitigate antenna violations by connecting floating metal nodes to a known voltage source (e.g., ground or power supply) using metal straps, diodes, or other structures. This prevents the accumulation of charge and minimizes the risk of damage during fabrication.

By performing thorough layout analysis for potential electrical issues such as short circuits, open circuits, and antenna violations, designers ensure the reliability and functionality of the IC. Early detection and resolution of these issues contribute to the successful fabrication and performance of the integrated circuit in real-world applications.

Post-Silicon Validation:

Prototype testing: Fabricate prototype silicon chips and conduct extensive testing to validate the design's functionality, performance, and reliability under real-world operating conditions. Carry out iterative refinements based on test results to ensure optimal efficiency.

Debugging: Identify and debug any discrepancies between expected and observed behavior using on-chip debugging tools, logic analyzers, and oscilloscopes.
Characterization: Characterize silicon performance by measuring key parameters such as speed, power consumption, and noise immunity across different operating conditions.

Throughout the testing process, documentation of test plans, test results, and design changes is essential for traceability and future reference. Iterative refinement of the design and testing procedures may be necessary to address any issues identified during testing and ensure the delivery of high-quality VLSI chips.

Furthermore, post-silicon validation plays a critical role in ensuring that the fabricated design aligns with pre-silicon simulation results. Any variations arising due to process, voltage, and temperature (PVT) conditions are carefully analyzed to understand their impact on circuit behavior. Advanced validation techniques, including stress testing and fault injection, are employed to evaluate the robustness of the system. This phase also helps in refining design margins and improving overall reliability before large-scale production.

In addition, feedback obtained during post-silicon validation is utilized to enhance future design iterations and optimize performance metrics. Design modifications based on real-time observations contribute to improving yield and reducing manufacturing defects. Collaboration between design and testing teams ensures efficient issue resolution and knowledge transfer. This continuous improvement cycle not only strengthens the current design but also provides valuable insights for developing more secure, efficient, and scalable VLSI systems in subsequent projects.

Moreover, integration of automated testing frameworks and advanced data analytics further enhances the efficiency of the validation process by enabling faster identification of anomalies and performance bottlenecks. This continuous improvement cycle not only strengthens the current design, leveraging machine learning techniques for pattern recognition in test data can significantly improve fault detection and predictive maintenance. Additionally, incorporating feedback from field performance monitoring allows engineers to validate real-world behaviour beyond controlled environments. This holistic approach ensures continuous optimization, accelerates time-to-market, and supports the development of highly reliable, power-efficient, and scalable VLSI solutions.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 EXECUTION RESULTS

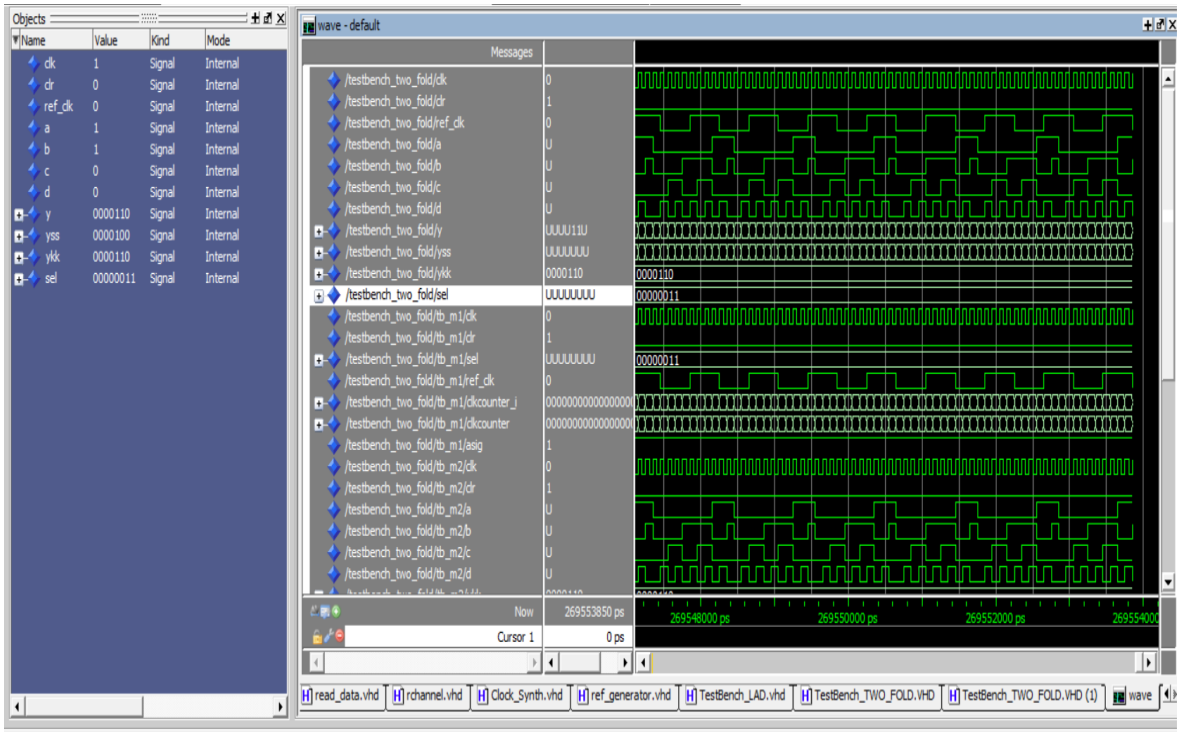

Fig. 8.1 Simulation Results of Synthesized Sweep Generator and Reference Clock Generator Outputs

Simulating a clock generator in ModelSim involves creating a clock signal that oscillates between logic levels (0 and 1) at a specified frequency. This is an essential step in digital design, as the clock drives the timing of the entire circuit. To simulate a clock generator in ModelSim, you typically write a testbench in VHDL or Verilog that includes a clock signal definition. The clock is generated by toggling a signal between high (logic 1) and low (logic 0) at regular intervals, using a process or an always block. For example, in Verilog, you might use a forever loop with a specified time delay to continuously flip the clock signal. Once the clock generator is defined in the testbench, you compile the testbench and run the simulation in ModelSim**.** During simulation, the generated clock signal can be observed using waveform viewers to verify its frequency, duty cycle, and stability. Designers can also introduce variations such as clock jitter, skew, or different frequencies to analyze system performance under diverse conditions.

8.1.1 CORRUPTION ATTACK

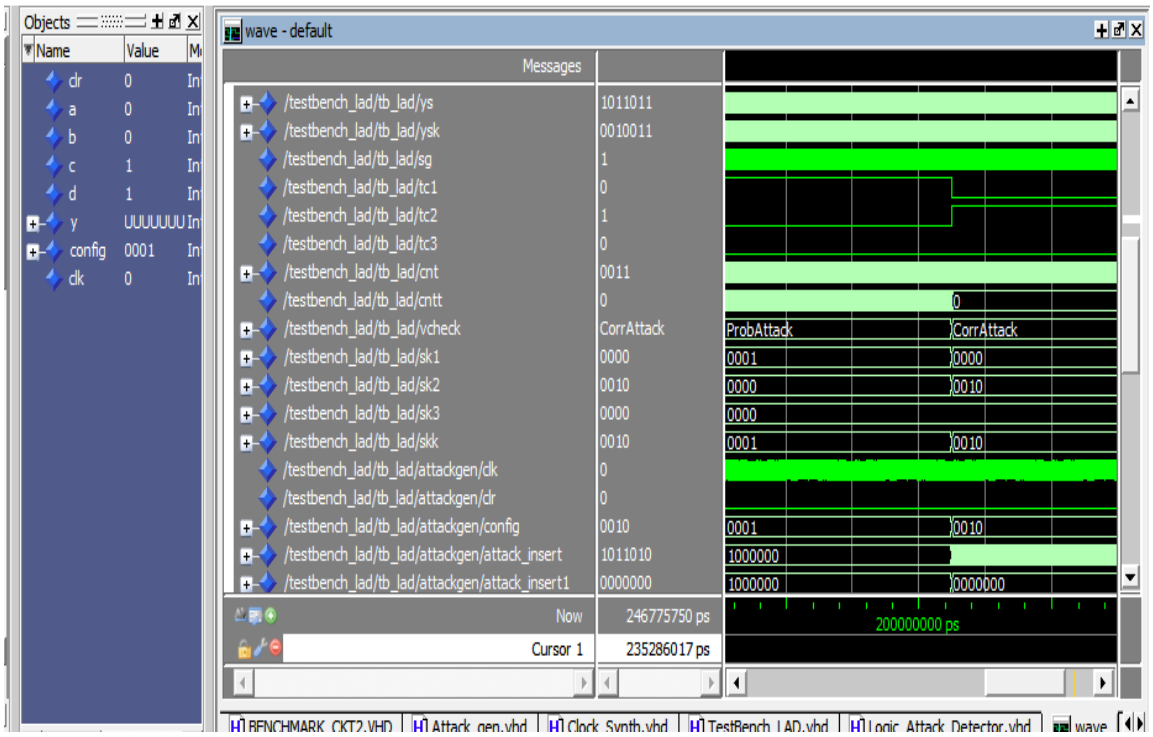

Fig. 8.2 Simulation Results of Corruption Attack Detection and Analysis

Detecting a corruption attack in a logic locking system through simulation in ModelSim involves verifying that the circuit's correct functionality has been compromised due to unauthorized tampering or attacks aimed at bypassing the locking mechanism. Logic locking is a hardware security technique used to protect intellectual property by adding extra logic gates or key inputs into the design, requiring a specific secret key to unlock the correct functionality of the circuit. In the case of a corruption attack, the attacker tries to alter the circuit, modify the key, or manipulate the design to gain access without the proper key.

To simulate the detection of such an attack in ModelSim, the locked circuit (along with its testbench) is created, and various key inputs, including incorrect and corrupted ones, are applied. The testbench would compare the outputs generated by the circuit for both valid and tampered keys to determine if the design exhibits faulty behaviour, such as incorrect logic states or unexpected outputs. ModelSim's waveform viewer helps in visually tracing how the circuit behaves in response to these corrupted inputs. The simulator can highlight abnormal transitions, glitches, or incorrect data propagation, signalling a successful attack or breach, this analysis assists in validating detection mechanisms and improving circuit.

Additionally, you can use assertions or self-checking mechanisms in the testbench to detect deviations from the expected output, ensuring that any discrepancies caused by an attack are flagged. By systematically running these tests under various scenarios, you can assess the vulnerability of the logic locking system and identify the exact conditions under which the corruption attack succeeds or fails. This simulation-based approach allows designers to refine the logic locking technique and strengthen the system's resistance to attacks before physical implementation.

8.1.2 PROBING ATTACK

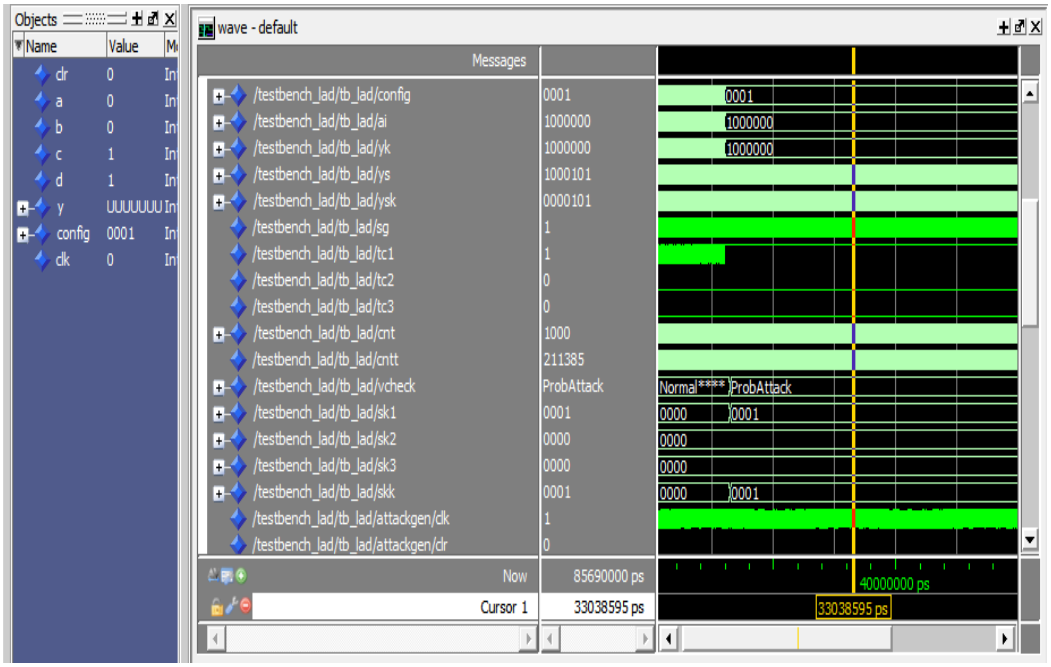

Fig. 8.3 Simulation Results of Probing Attack Detection and Analysis

Detecting a probing attack in a logic locking system through simulation in ModelSim involves monitoring the circuit's response to potential attempts by an attacker to extract sensitive internal signals or key bits. A probing attack targets the internal workings of the hardware, trying to directly observe the values of signals at critical points to reverse-engineer the locking mechanism or determine the secret key. Logic locking systems protect hardware by requiring the correct key for proper functionality, and probing attacks attempt to bypass this by accessing internal information instead of brute-forcing the key. Such simulations help identify vulnerable nodes and enable the design of stronger countermeasures to prevent unauthorized signal observation.

In ModelSim, the detection of such an attack can be simulated by creating a testbench that introduces scenarios where internal nodes are probed or manipulated. For instance, you can model the attacker's access to specific wires or nodes and observe how the system reacts. The testbench might simulate both normal operation (with a correct key) and tampered scenarios, where internal signals are artificially accessed or forced to erroneous values. By examining the circuit's response through the waveform viewer or by using assertions that check for unexpected behavior (such as incorrect outputs or unusual signal transitions), you can identify whether probing attempts are detectable.

Furthermore, you can incorporate tamper detection mechanisms or countermeasures in the design, such as randomization of internal signals, noise injection, or logic obfuscation, to confuse or mislead an attacker. These countermeasures can also be simulated in ModelSim to evaluate their effectiveness. The simulation helps assess how well the logic locking system resists probing attacks and whether the detection mechanisms successfully identify unauthorized access to internal signals, ensuring the robustness of the design against this form of attack before hardware deployment

8.1.3 FORBIDDEN ATTACK

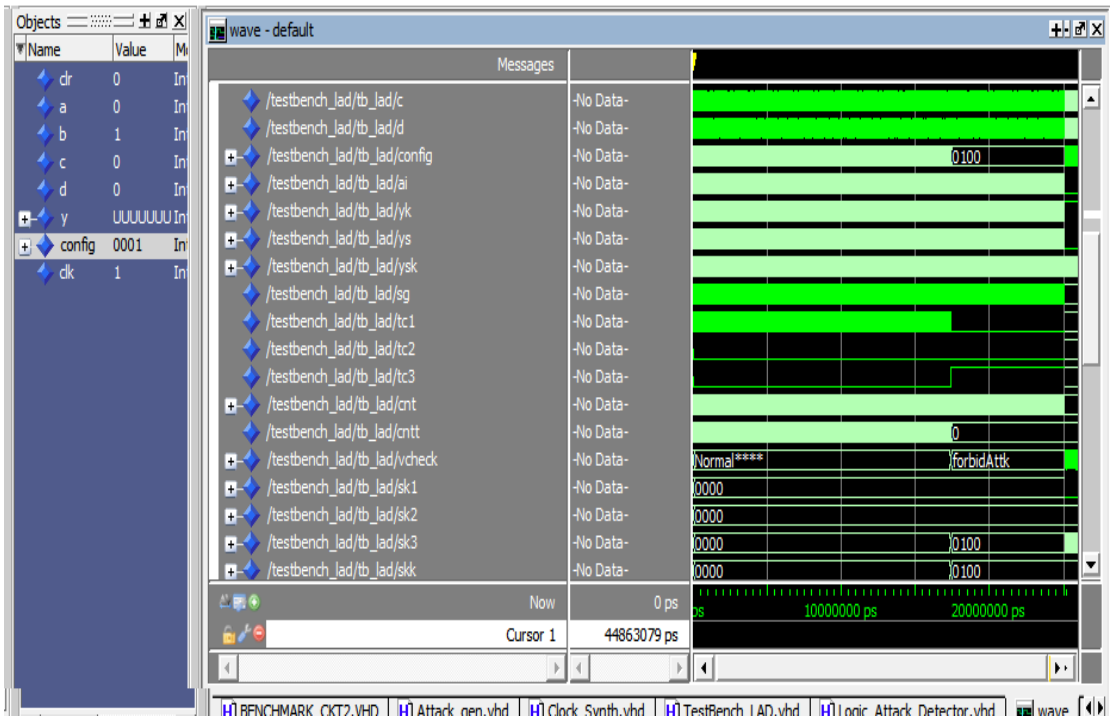

Fig. 8.4 Simulation Results of Forbidden Attack Detection and Analysis

Detecting a forbidden attack in a logic locking system through simulation in ModelSim involves identifying attempts by an attacker to exploit the system by forcing inputs or specific signals into "forbidden" or illegal states to bypass the locking mechanism. In a logic locking system, certain key inputs or signal combinations are designed to trigger incorrect or unintended functionality if not applied correctly. A forbidden attack manipulates these inputs, aiming to uncover vulnerabilities or trick the circuit into functioning without the correct key.

To simulate and detect such an attack in ModelSim, the design is locked, and a testbench is created that systematically applies invalid or forbidden key combinations to the circuit. The testbench checks whether the logic locking system reacts appropriately to these illegal inputs—such as producing erroneous outputs or entering a defined failure mode. Using ModelSim's waveform viewer, designers can visually inspect how the circuit responds to forbidden key inputs, identifying unusual behaviors like unintended data propagation, incorrect state transitions, or failure to lock the circuit as expected.

Moreover, assertions or self-checking mechanisms can be implemented in the testbench to automatically flag deviations from the expected behavior, such as incorrect outputs or system malfunctions, indicating a successful forbidden attack. By simulating various attack scenarios, including forcing inputs outside the allowed range or probing the circuit's responses to invalid combinations, ModelSim helps to identify whether the logic locking system is vulnerable to forbidden attacks.

8.1.4 NORMAL PROCESS

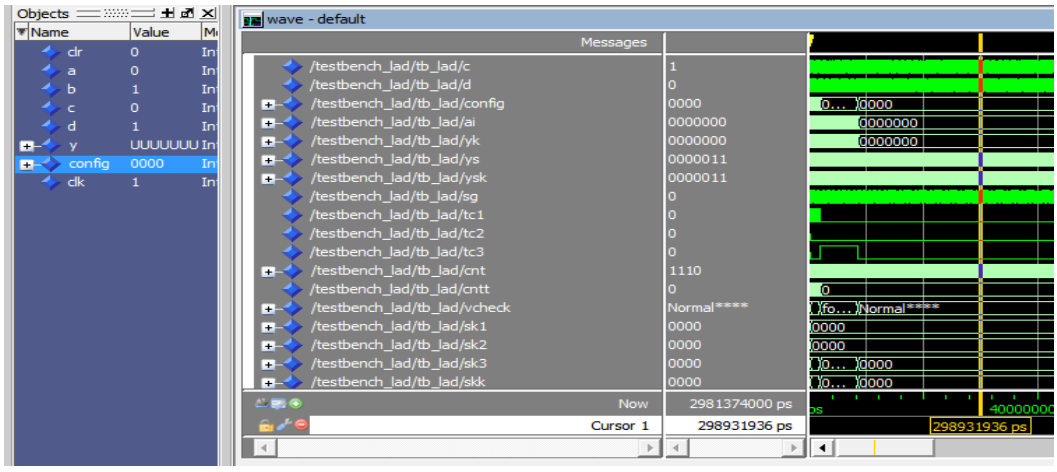

Fig. 8.5 Simulation Results of Normal Process

The normal process represents the standard operational behavior of the proposed system under attack-free conditions. In this phase, all modules such as the reference clock generator, clock synthesizer, benchmark circuits, and detection units function as intended without any external interference. The generated clock signals maintain stable frequency and timing characteristics, ensuring proper synchronization across all components. The system processes input signals accurately, and the output responses match the expected functional behavior, validating the correctness of the design and its implementation.

During this process, the detection modules remain in an idle monitoring state, continuously observing system parameters such as voltage levels, timing variations, and signal transitions. Since no attack patterns are introduced, the system does not exhibit any anomalies or irregularities. This confirms that the detection framework does not produce false positives under normal conditions. The successful execution of the normal process establishes a reliable baseline for comparison with various attack scenarios, thereby demonstrating the stability, accuracy, and robustness of the overall system design.

8.2 DATAFLOW DIAGRAMS

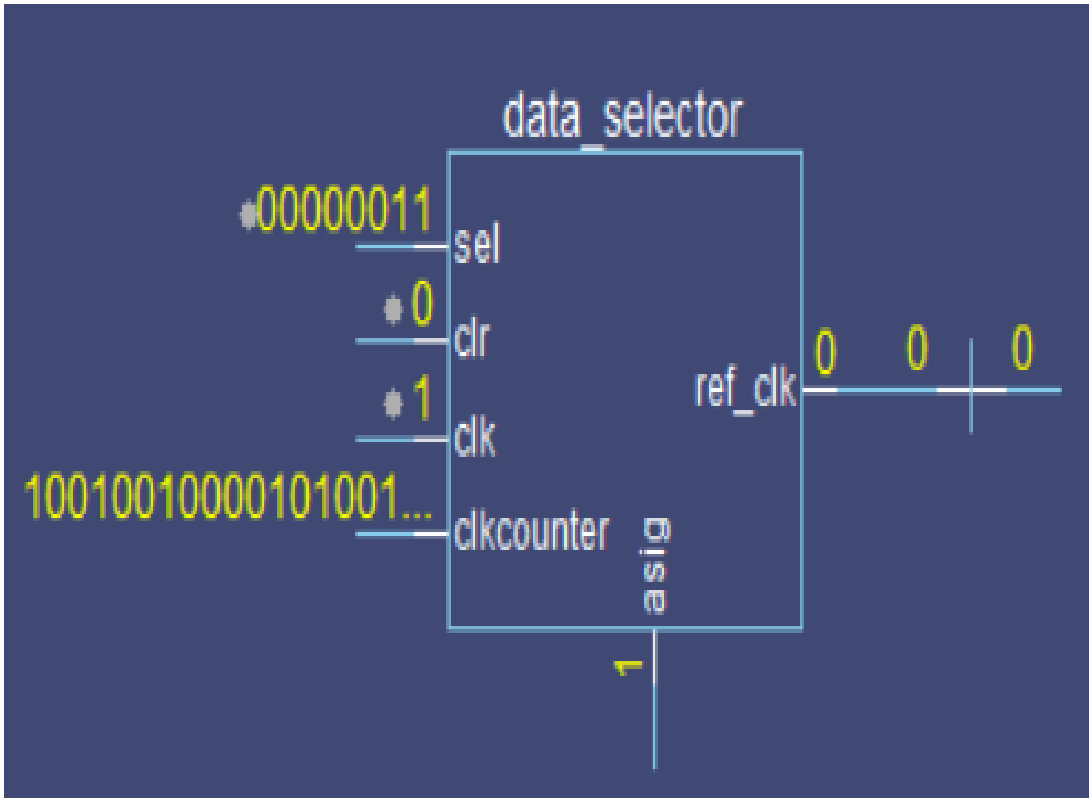

Fig. 8.6 Dataflow diagram of Testbench – External view

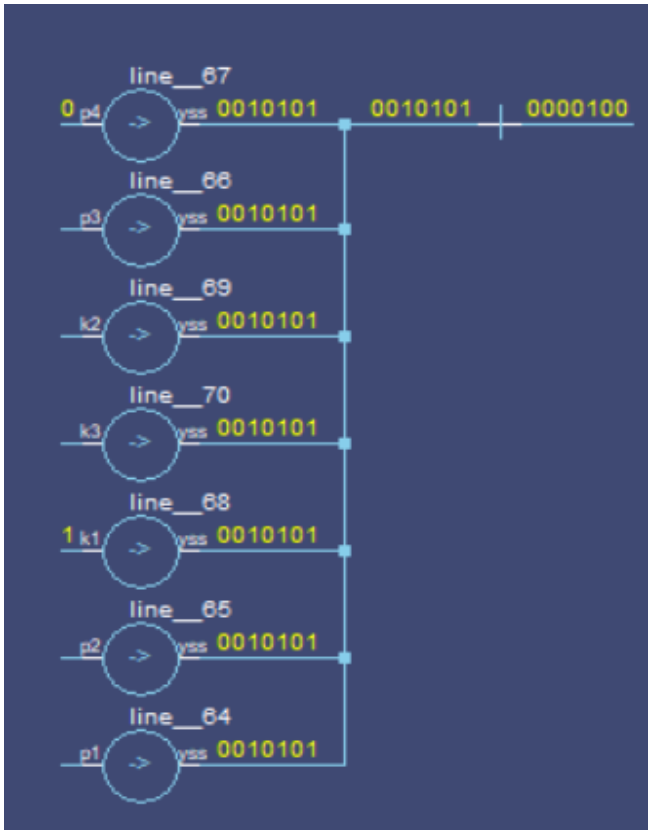

Fig. 8.7 Dataflow diagram of Testbench – Internal Integration

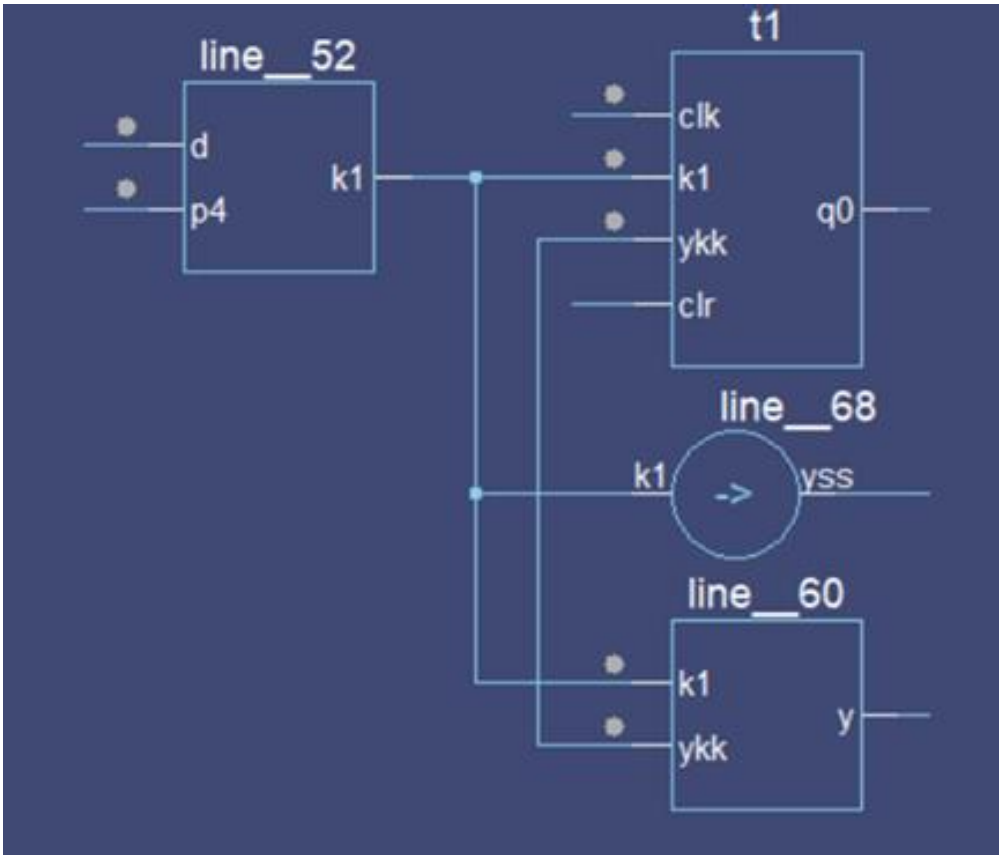

Fig. 8.8 Dataflow diagram of Attack Pattern Generator

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 CONCLUSION

Side-channel attacks pose significant threats to System on Chip (SoC) platforms, particularly affecting the operations of FPGAs. The increasing integration of diverse functionalities onto a single silicon platform in modern chip designs has heightened the need for robust security measures, particularly encryption. Any side-channel attacks targeting FPGA devices can compromise the system's integrity by manipulating logic operations, leading to unintended data leaks from external sources. In the proposed system, specialized test circuits are developed to closely monitor and analyze the impact of side-channel attacks. The system model features a configurable high-speed clock switching network that operates with multiple clock sources to simulate various attack scenarios. Additionally, a differential attack generator is integrated to introduce side-channel, corruption, and FPGA replay attacks into the application circuit for testing.

To enhance the reliability of the proposed framework, comprehensive simulation and verification are carried out using ModelSim, ensuring accurate analysis of circuit behavior under both normal and attack conditions. The modular design approach allows seamless integration of clock generation, attack injection, and detection units, thereby improving scalability and flexibility of the system. By observing waveform outputs and timing responses, the system effectively identifies anomalies introduced by various attack patterns, enabling precise detection and analysis of potential security breaches within FPGA-based SoC environments, there is significant scope for further enhancement and extension of this work.

The system's performance is evaluated continuously through a Recurrent Pattern Verification (RPV) Algorithm, which classifies the effects of each attack. The evaluation includes metrics such as power consumption, latency, and device utilization. Cryptographic principles and protective frameworks are typically employed to secure most SoC platforms, but under induced attacks, a leakage power of 0.009 watts was observed, demonstrating the system's vulnerability and the importance of countermeasures.

9.2 FUTURE WORK

The proposed system demonstrates an effective framework for detecting and analyzing side-channel and physical attacks in FPGA-based SoC designs. However, there is significant scope for further enhancement and extension of this work. One potential direction is the integration of machine learning and deep learning techniques for intelligent attack detection and classification. By leveraging data-driven models, the system can achieve higher accuracy in identifying complex and previously unseen attack patterns, thereby improving overall security robustness.

Additionally, the system can be extended to support a wider range of advanced attack models, including electromagnetic attacks, thermal attacks, and fault injection techniques. Incorporating real-time hardware implementation and validation on physical FPGA platforms would further strengthen the practical applicability of the proposed design. Future work can also focus on optimizing power consumption, area overhead, and latency to make the system more efficient for deployment in resource-constrained environments. Furthermore, the integration of adaptive and self-healing security mechanisms can enhance the resilience of SoC systems against evolving hardware threats, making the design more robust and scalable for next-generation secure embedded applications.

Another potential area for future work involves extending the proposed framework to support heterogeneous system architectures, including multi-core and distributed SoC environments. As modern systems increasingly rely on interconnected modules, ensuring secure communication between components becomes critical. Incorporating secure communication protocols and hardware-level encryption techniques can further enhance system integrity. Additionally, exploring compatibility with emerging technologies such as IoT and edge computing devices can broaden the applicability of the proposed system across diverse real-world scenarios.

Further improvements can also be achieved by developing automated design and verification methodologies using advanced EDA tools. Integrating formal verification techniques alongside simulation-based approaches can ensure higher design correctness and reduce the chances of undetected vulnerabilities. Moreover, implementing scalable design architectures that can adapt to different technology nodes will improve long-term usability.

REFERENCES

[1] Y. Ji and E. Dubrova, (2025) "A side-channel attack on a masked hardware implementation of CRYSTALS-Kyber", Journal of Cryptographic Engineering, vol. 15, no. 7, pp. 1-23, doi: 10.1007/s13389-025-00375-7.

[2] A. Srivastava, S. S. Miftah, H. Kim, D. Pal, and K. Basu, (2025) "PoSyn: Secure Power Side-Channel Aware Synthesis", arXiv preprint, arXiv:2506.08252v1, pp. 1-13.

[3] D. Xu, K. Wang, and J. Tian, (2025) "A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber", IEEE Transactions on Circuits and Systems-II: Express Briefs, vol. 72, no. 3, pp. 504-508, doi: 10.1109/TCSII.2024.3407024.

[4] Jiheon Woo, Donggyun Ryu, Daewon Seo, Young-Sik Kim, Namyoon Lee, Yuval Cassuto, and Yongjune Kim, (2026) " Mutual Information Minimization for Side-Channel Attack Resistance via Optimal Noise Injection", arXiv preprint, arXiv:2504.20556v4, pp. 1-15.

[5] M. Yasin and O. Sinanoglu, (2017) "Evolution of logic locking", IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC), pp. 1-6, doi: 10.1109/VLSI-SoC.2017.8203496.

[6] M. Rostami, F. Koushanfar and R. Karri, (2014) "A Primer on Hardware Security: Models, Methods, and Metrics," in Proceedings of the IEEE, vol. 102, no. 8, pp. 1283-1295, Aug. 2014, doi: 10.1109/JPROC.2014.2335155.

[7] T. Thangam, G. Gayathri and T. Madhubala, (2017) "A novel logic locking technique for hardware security", 2017 IEEE International Conference on Electrical, Instrumentation

and Communication Engineering (ICEICE), 2017, pp. 1-7, doi: 10.1109/ICEICE.2017.8192439.

[8] H. -Y. Chiang, Y. -C. Chen, D. -X. Ji, X. -M. Yang, C. -C. Lin and C. -Y. Wang,(2020) "LOOPLock: Logic Optimization-Based Cyclic Logic Locking" in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, no. 10, pp. 2178-2191, , doi: 10.1109/TCAD.2019.2960351.

[9] D. Sirone and P. Subramanyan, (2019)"Functional Analysis Attacks on Logic Locking," Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 936-939, doi: 10.23919/DATE.2019.8715163.

[10] Rahman, M.T., Rahman, M.S., Wang, H., Tajik, S., Khalil, W., Farahmandi, F., Forte, D., Asadizanjani, N., & Tehranipoor, M.M. (2020). Defense-in-Depth: A Recipe for Logic Locking to Prevail. ArXiv, abs/1907.08863.

[11] Jain, A., Zhou, Z. and Guin, U., (2021). TAAL: Tampering Attack on Any Key-based Logic Locked Circuits. ACM journals, 26(4), pp.1-22.

[12] X. Xu, B. Shakya, M. M. Tehranipoor, and D. Forte, (2017) "Novel bypass attack and BDD-based tradeoff analysis against all known logic locking attacks," in Proc. Int. Conf. Cryptograph. Hardw. Embedded Syst. Cham, Switzerland: Springer, pp. 189–210.

[13] H. M. Kamali, K. Z. Azar, H. Homayoun, and A. Sasan, (2019) "Full-lock: Hard distributions of SAT instances for obfuscating circuits using fully configurable logic and routing blocks" in Proc. 56th Annu. Design Automat. Conf., pp. 1–6.

[14] K. Shamsi, M. Li, T. Meade, Z. Zhao, D. Z. Pan, and Y. Jin, (2017) "Cyclic obfuscation for creating SAT-unresolvable circuits" in Proc. Great Lakes Symp. VLSI, pp. 173–178.

[15] S. Roshanisefat, H. M. Kamali, and A. Sasan,(2018)“SRCLock: SAT-resistant cyclic logic locking for protecting the hardware” in Proc. Great Lakes Symp. VLSI, May 2018, pp. 153–158.

[16] M. Yasin, A. Sengupta, M. T. Nabeel, M. Ashraf, J. Rajendran, and O. Sinanoglu, (2017) “Provably-secure logic locking: From theory to practice” in Proc. ACM SIGSAC Conf. Comput. Commun. Secur, pp. 1601–1618.